

Learning Long-Range Dependencies with Temporal Predictive Coding

Tom Potter^{a,*}, Oliver Rhodes^a

^a*International Centre for Neuromorphic Systems, The University of Manchester, Manchester, United Kingdom*

Abstract

Temporal Predictive Coding provides a layer-local, parallelisable mechanism for learning in recurrent systems, making it an attractive candidate for online local learning on neuromorphic and edge hardware. However, its recurrent parameter update captures only local temporal relationships, neglecting the historic influence of parameters along the latent-state trajectory, and therefore struggles to assign credit over longer temporal horizons. This work combines for the first time Temporal Predictive Coding with Real-Time Recurrent Learning (tPC-RTRL), incorporating an online influence matrix that tracks this historic effect whilst preserving the spatial and temporal locality properties valued by neuromorphic implementations. Under explicit assumptions, we prove that tPC-RTRL recovers the gradients of backpropagation-through-time exactly. Empirically, a near-equivalence holds across several tasks of varying scale and complexity, including byte-level language modelling on WikiText-103 (tPC-RTRL vs. BPTT: 1.865 vs. 1.864 validation BPC), English–French translation on a CCMatrix subset (20.23 vs. 20.29 BLEU), and a realistic nanodrone system-identification benchmark (0.506 m vs. 0.505 m mean position error). Finally, we show that the iterative inference mechanism used during training can be reused at deployment time to incorporate intermittent state observations, halving final-position error relative to open-loop rollout on the nanodrone task (0.402 m vs. 0.805 m) and suggesting a path towards unifying learning and filtering within the same computational framework.

Keywords: predictive coding, recurrent neural networks, temporal credit assignment, real-time recurrent learning, online filtering

1. Introduction

Backpropagation of errors (BP) [1] has long served as the dominant algorithm for training artificial neural networks, powering breakthroughs in computer vision [2], natural language processing [3], and reinforcement learning [4]. However, as models continue to grow to billions of parameters, training them is becoming prohibitively expensive, both computationally and energetically. This has led to growing concerns about the environmental impact and long-term sustainability of the field [5], and has prompted the search for learning algorithms with lower resource demands.

Motivated by the efficiency of biological intelligence, neuroscience-inspired alternatives to BP have

started to gain traction. The brain operates under strict resource constraints, reasoning and learning with a fraction of the energy of its artificial counterparts. Inspired by this, several recent works have explored the potential of biologically inspired learning algorithms as a pathway towards more efficient learning systems [6–8], many of which are attractive from a hardware perspective due to their compatibility with the design principles of neuromorphic and edge computing platforms [9].

Predictive Coding (PC) [10] is one such promising alternative. With roots in theories of cortical function [11, 12], information theory [13, 14], and ties to variational inference [15–17], PC presents a method of credit assignment consisting of local, parallelisable operations, making it well suited to hardware where inter-core communication dominates energy cost, on-chip memory is limited, and strict global synchronisation is expensive [18]. Recent work has shown that PC can closely approximate BP-derived up-

*Corresponding author

Email addresses: `thomas.potter@manchester.ac.uk` (Tom Potter), `oliver.rhodes@manchester.ac.uk` (Oliver Rhodes)

dates [19, 20] and, under explicit conditions, match them exactly [21, 22]. When these exact-equivalence constraints are relaxed, PC has been associated with further desirable properties including improved sample efficiency, robustness, and advantages in online settings [8], with recent analyses suggesting that PC-derived updates can capture aspects of higher-order optimisation [23].

Despite this progress, PC approaches remain less developed in recurrent and temporally extended settings. Although several works have explored recurrent PC formulations [24–27], they have typically only been evaluated on relatively small-scale tasks [26, 27] and often struggle with long-range temporal credit assignment [24, 25].

To address these limitations, this paper explores Temporal Predictive Coding with Real-Time Recurrent Learning (tPC-RTRL): a PC algorithm for recurrent neural networks that combines the local predictive-error computations of PC, with an RTRL-based online recurrent credit-assignment mechanism. We begin by highlighting a specific, structural limitation of standard Temporal Predictive Coding (tPC), identifying that its parameter update rule neglects the historic influence of the recurrent parameters throughout the latent-state trajectory (Section 2.3). We then introduce tPC-RTRL as a natural solution to this problem aiming to improve the model’s ability to learn temporal relationships (Section 3.2). Next, we show that, under explicit assumptions, this algorithm recovers BPTT gradients exactly (Section 3.4), supporting the idea that tPC-RTRL has the ability to recreate the strong performance of BPTT on a range of tasks. Finally, we provide empirical evidence for this claim, demonstrating that tPC-RTRL matches the performance of BPTT on a range of sequence learning tasks (Section 4).

In summary, this work makes the following contributions:

- Introduces Temporal Predictive Coding with Real-Time Recurrent Learning (tPC-RTRL).
- Shows that, under explicit assumptions, tPC-RTRL recovers the gradients computed by backpropagation-through-time (BPTT), providing a theoretical validation that BPTT-derived updates can be recovered in a local, parallelisable manner.
- Demonstrates the effectiveness of tPC-RTRL on both synthetic and real-world sequence-learning tasks, including language modelling

on WikiText-103 [28] and translation on an English–French subset of CCMatrix [29].

- Demonstrates that the same iterative inference mechanism used during tPC-RTRL training can be reused for test-time state correction on a real-world nanodrone benchmark [30].

2. Background

This section provides background to support the methods developed in Section 3. We begin by motivating the search for alternatives to backpropagation (BP) through the lens of neuromorphic hardware (Section 2.1). We then review the Predictive Coding (PC) framework (Section 2.2) underlying our approach, with a particular focus on Temporal Predictive Coding (tPC, Section 2.3). Finally, we introduce Real-Time Recurrent Learning (RTRL, Section 2.4), the recurrent credit-assignment mechanism we later use to extend tPC to longer-range temporal learning.

2.1. Neuromorphic hardware and on-chip learning

Neuromorphic hardware offers a promising alternative to conventional accelerators for applications requiring low-power, low-latency intelligence, operating continually at the edge. Rather than concentrating computation in a small number of large processing units, neuromorphic systems distribute state, memory, and computation across many small cores. Each core typically maintains local neuron and synapse state and exchanges information primarily with synaptically connected units. The low-power potential of this computing paradigm is attractive for several applications, for example in always-on acoustic or visual sensing, wearable monitoring, autonomous robotics, drones, and industrial monitoring. In such settings, transmitting raw data to a remote accelerator or repeatedly retraining a model offline may be undesirable because of energy, latency, bandwidth, or privacy constraints [18, 31, 32].

For inference, neural networks can often be mapped relatively directly onto such substrates. Neural state can be maintained locally, and activations are communicated along the same synaptic connections that define the model. However, training these models on-chip is substantially more difficult. BP, the algorithm used to train modern neural networks, consists of a forward computation followed by a distinct reverse-mode computation. The forward pass computes neural activations, whereas the

backward pass must propagate error derivatives in the opposite direction through the network, combine them with the local activation derivatives, or stored neuron state, and use the resulting signals to update each parameter. Conventional accelerators are designed to support this process. Intermediate activations can be stored in large global memories, and the same dense linear-algebra units used during the forward pass can be reused during a globally coordinated, layer-by-layer reverse traversal of the model’s compute graph. In contrast, neuromorphic substrates are primarily designed to update local neural state, in response to incoming synaptic transmissions. For the system to support BP, it must additionally represent and route error signals, retain or reconstruct the activities and derivatives needed for updates, and schedule this process using an explicit coordination or scheduling mechanism.

Much of the on-chip neuromorphic-learning literature reflects these problems. A common approach is to train a network off-chip using BP, then deploy the learnt parameters to neuromorphic hardware for inference. Hardware-in-the-loop methods take a partial step towards adaptation on the device, running the network on the neuromorphic substrate, but reading state from the chip to compute parameter updates externally using conventional methods [33]. Other approaches avoid BP altogether, using local plasticity rules such as spike-timing-dependent plasticity, reward-modulated plasticity, or eligibility-trace-based algorithms such as e-prop [34–36]. These methods support on-chip and online adaptation using signals that can be computed and routed locally, but make different trade-offs between learning performance, and hardware compatibility.

Recent work by Renner et al. [37] demonstrates that exact BP can nevertheless be implemented fully on-chip on Intel’s Loihi processor [9]. However, this result does not just map a conventional feedforward network onto the hardware. The implementation augments the feedforward pathway with multiple additional components, including a dedicated error-propagation pathway, relay populations that preserve activity for subsequent updates, and a control circuit that coordinates the successive stages of forward propagation, error computation, backward propagation, and weight updates. This is an important step towards on-chip BP, but it also highlights the accommodations that must be made to implement BP on this type of hardware, and the overheads that are incurred in doing so. The challenge becomes more pronounced in recurrent

systems. BPTT accounts not only for how an error propagates across layers, but also for how parameters influenced the current state through the entire recurrent state trajectory. It does this by storing intermediate activations to accommodate unrolling the recurrent network through time and traversing the unrolled computation in reverse. Training recurrent networks on-chip requires new ways of thinking about temporal credit assignment that avoid the need to store and traverse the full latent-state trajectory [36].

Predictive Coding (PC) offers a potential algorithm which is more compatible with the design principles of neuromorphic hardware. Rather than separating inference from a distinct reverse-mode phase, PC represents prediction errors within the network and iteratively updates latent states to minimise a shared free energy. The local dynamics that minimise this free energy also disperse prediction errors throughout the network. Parameter updates can then be computed from locally available activities and prediction errors. Therefore, the potential advantage of PC is that credit assignment and inference are embedded in the same dynamical process, rather than requiring a separately scheduled global backward computation. Furthermore, although PC inference is described as gradient descent on a free energy objective, it can be implemented through local synaptic interactions, aligning much more closely with the preferences of neuromorphic hardware than the global, reverse-mode traversal of BP. This perspective has led to a growing interest in PC as a low-resource alternative to BP, and motivates PC for on-chip learning.

2.2. Predictive Coding

Predictive Coding (PC) originated as a theory of hierarchical cortical processing, in which each layer predicts the activity of the layer below and only the residual prediction error is propagated forward [10–12]. Over time, this idea has been connected to information-theoretic accounts of efficient coding [13, 14], variational free energy formulations of perception and learning [15–17], and more recently to machine-learning views of local credit assignment [19, 21]. From this perspective, PC provides not only a descriptive theory of neural computation, but also a practical algorithmic framework in which inference and learning are both driven by the minimisation of local prediction errors.

PC as a Machine Learning Algorithm. When viewed as a machine-learning algorithm, PC performs approximate Bayesian inference in a hierarchical generative model by minimising a variational free energy $\mathcal{F}$ [15, 38]. Under Gaussian conditionals and a mean-field approximation in which each variational factor is sharply peaked around its current state estimate, $\mathcal{F}$ reduces to a sum of local squared prediction errors,

$$\mathcal{F} = \frac{1}{2} \sum_{l=0}^{L-1} \epsilon_l^\top \epsilon_l, \quad \epsilon_l = \mathbf{x}_l - g_l(\mathbf{x}_{l+1}; \theta_l), \quad (1)$$

where g_l denotes the downward prediction function from layer $l+1$ to layer l with parameters θ_l , and $\mathbf{x}_0 \equiv \mathbf{y}$ is the observed target. PC is typically implemented via an Expectation-Maximisation (EM)-style procedure [19]. An E-step iteratively updates the latent activities $\mathbf{x}_{1:L-1}$ to reduce $\mathcal{F}$ with parameters held fixed, followed by an M-step that updates parameters by gradient descent on the same objective with activities held fixed. The resulting inference and learning rules are layer-local, with each update depending only on the prediction error at its own layer and the layer directly below. A full derivation, including the variational free energy setup and the conditions under which the Hebbian-like parameter update arises, is given in Appendix A.

A growing body of work has shown that PC can closely approximate BP in feedforward networks, recover exact BP gradients under suitable assumptions, and extend beyond layered architectures to more general computation graphs [19–22, 39]. This has shifted PC from being viewed purely as a neuroscientific theory of cortical function towards a promising framework for local credit assignment. At the same time, recent analyses suggest that when exact BP-equivalence constraints are relaxed, PC exhibits desirable optimisation behaviour, including robustness in online settings and the ability to incorporate higher-order curvature information [8, 23].

Recent work has shown that practical performance can depend strongly on implementation choices [40]. A growing line of work has begun to study the optimisation and architectural details required to train PC networks reliably, with the aim of developing a repertoire of best practices analogous to those that have contributed to the success of BP. This includes work on inference and parameter optimisers [41] as well as architectural choices such as activation functions [42], and modifications to the inference and learning rules to improve training in deeper

networks [43].

2.3. Temporal Predictive Coding

Recurrent Predictive Coding. Several lines of work have extended PC to temporally structured data. One family introduces temporal structure through generalised coordinates of motion, in which hidden states are augmented with higher-order temporal derivatives [17, 38]. These approaches provide a principled account of temporal inference and generalised filtering, but can increase modelling and numerical complexity as progressively higher-order derivatives are retained, and may be sensitive to discretisation error and noisy observations. More recently, dynamic PC models have explored hierarchical spatiotemporal prediction in which higher-level latent variables modulate lower-level transition dynamics [26]. These works demonstrate that PC can support rich temporal structure, but they often rely on specialised architectural mechanisms to parameterise dynamics, making it less clear how temporal credit assignment should be performed in standard recurrent networks.

Temporal Predictive Coding (tPC). Temporal Predictive Coding extends PC to sequential data by formulating sequence processing as inference in a recurrent system [24, 25]. In this view, the goal is not only to predict future observations, but to infer the latent state of the system as observations arrive over time. Millidge et al. [24] formulate this problem using a hidden Markov model with control inputs, in which a latent state evolves according to recurrent dynamics and generates noisy observations. An important feature of this formulation is that it connects PC to classical state estimation. In the linear-Gaussian case, the resulting iterative inference dynamics recover hidden-state tracking behaviour closely related to Kalman filtering, but without explicitly propagating posterior covariance estimates [24]. This makes tPC especially appealing in partially observed settings, where sequential prediction and online state inference must be performed simultaneously within a single recurrent dynamical system. Beyond this, Tang et al. [25] study tPC for sequential memory, showing that the approach can memorise and retrieve sequential inputs accurately with a biologically motivated implementation.

Next, we introduce the tPC framework [24] which serves as the starting point of our method. Consider a tPC network, where the input at timestep t is

clamped as $\mathbf{x}_L^{(t)}$, the target is denoted $\mathbf{y}^{(t)}$, and a latent state $\mathbf{x}^{(t)}$ must be inferred. The algorithm itself makes minimal specifications about the structure of the network; the key conceptual move is from a static PC network with no temporal relationships, to one in which previous latent states provide some form of prior over inference at the current timestep. For clarity, we first describe the single-latent-state case; the multilayer extension follows directly and is given in [24].

The temporal generative model factorises as

$$p(\mathbf{y}^{(t)}, \mathbf{x}^{(t)} \mid \mathbf{x}_L^{(t)}, \hat{\mathbf{x}}^{(t-1)}; \theta) = p(\mathbf{y}^{(t)} \mid \mathbf{x}^{(t)}; \theta_{\text{out}}) p(\mathbf{x}^{(t)} \mid \mathbf{x}_L^{(t)}, \hat{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}}), \quad (2)$$

where $\hat{\mathbf{x}}^{(t-1)}$ denotes the converged latent-state estimate from the previous timestep. Thus, inference at time t must balance finding a latent state that explains the current target whilst remaining consistent with a temporal prediction based on the previous state and current input. Assuming Gaussian conditionals and the sharply-peaked variational approximation used in the static case (Appendix A), the instantaneous free energy reduces to

$$\mathcal{F}^{(t)} = \frac{1}{2} \|\epsilon_x^{(t)}\|^2 + \frac{1}{2} \|\epsilon_y^{(t)}\|^2, \quad (3)$$

where

$$\epsilon_x^{(t)} = \mathbf{x}^{(t)} - f(\mathbf{x}_L^{(t)}, \hat{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}}), \quad (4)$$

$$\epsilon_y^{(t)} = \mathbf{y}^{(t)} - g(\mathbf{x}^{(t)}; \theta_{\text{out}}). \quad (5)$$

We refer to the functions f and g as the predictive prior and decoder, respectively. These functions generate predictions of neighbouring variables, which we may represent with

$$\mu_x^{(t)} = f(\mathbf{x}_L^{(t)}, \hat{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}}), \quad (6)$$

$$\mu_y^{(t)} = g(\mathbf{x}^{(t)}; \theta_{\text{out}}). \quad (7)$$

This decoder function is sometimes called the read-out, as it maps the recurrent latent state to the output space. It could also be a multilayer network, in which case multiple intermediate latent states may be inferred as in a static PC network.

As in Section 2.2, tPC is implemented by alternating iterative inference (E-step) and parameter updates (M-step) at each timestep. The resulting update rules preserve the locality of standard PC, with the addition of a predictive prior through the previous inferred state, given by f .

A structural limitation of standard tPC. This formulation has an important consequence for the recurrent parameters. The instantaneous free energy $\mathcal{F}^{(t)}$ depends on θ_{rec} only through the current transition prediction $\mu_x^{(t)}$. The previous inferred state $\hat{\mathbf{x}}^{(t-1)}$ is treated as a fixed variable, rather than as a function of θ_{rec} . The recurrent parameter update therefore takes the form

$$\Delta\theta_{\text{rec}} = -\eta \frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} = -\eta \frac{\partial \mathcal{F}^{(t)}}{\partial \mu_x^{(t)}} \frac{\partial \mu_x^{(t)}}{\partial \theta_{\text{rec}}^{(t)}}, \quad (8)$$

where $\theta_{\text{rec}}^{(t)}$ denotes the application of the recurrent parameters at time t and $\mu_x^{(t)}$ denotes the recurrent prediction given by the transition prior. This update captures only the immediate influence of θ_{rec} on the current recurrent prediction. For parameters that are not reused through the latent-state trajectory, such as θ_{out} , this is sufficient. For recurrent parameters, however, it neglects their historic influence through earlier propagated states, and therefore cannot reliably propagate recurrent parameter credit across more than a single timestep. In this respect, standard tPC has the same temporal credit-assignment structure as one-step truncated BPTT. Section 4.1 gives an empirical demonstration of this equivalence. This missing temporal credit-assignment term is what motivates the RTRL-based approach developed in Section 3.

2.4. Real-Time Recurrent Learning

Williams and Zipser [44] introduced Real-Time Recurrent Learning (RTRL) as an alternative to backpropagation-through-time (BPTT) for recurrent neural networks. Unlike BPTT, RTRL does not require the network to be unrolled through the full input sequence, and its memory cost is therefore independent of sequence length. Instead, temporal credit assignment is performed online by maintaining an influence matrix that tracks how the current hidden state depends on the recurrent parameters.

Figure 1 highlights the key differences between BPTT and RTRL. BPTT must store the whole latent-state trajectory, and computes parameter updates by stepping backwards through the unrolled computation graph. RTRL instead maintains an online influence matrix that captures the effect successive applications of the recurrent parameters have had on the current hidden state. This allows RTRL to compute exact recurrent parameter gradients

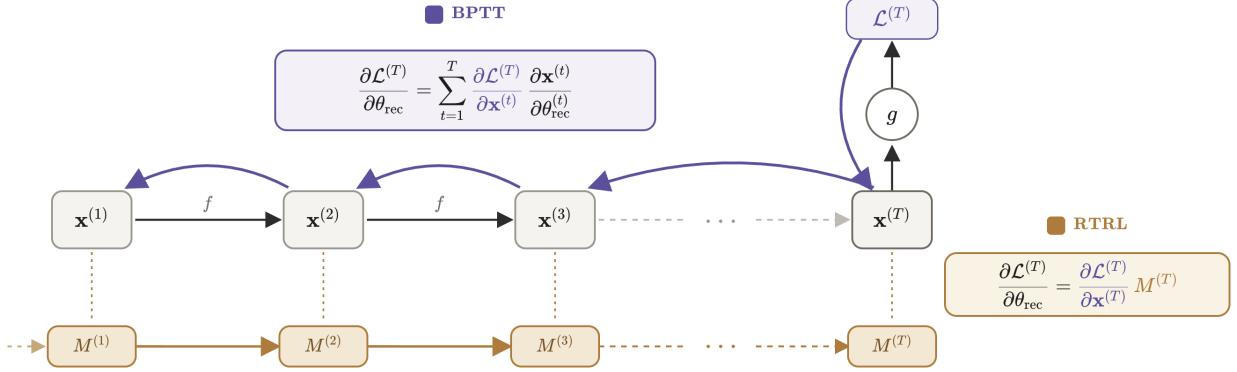


Figure 1: Comparison of temporal credit assignment in backpropagation-through-time (BPTT) and Real-Time Recurrent Learning (RTRL). BPTT computes recurrent parameter gradients by propagating loss derivatives backward through an unrolled state trajectory. RTRL instead maintains an online influence matrix during the forward execution of the network, allowing the current loss gradient to incorporate the historic influence of recurrent parameters without a reverse traversal of the full sequence. RTRL does not change how spatial credit assignment is performed.

without needing to store or traverse the full latent-state trajectory. RTRL trades the cost of maintaining this influence matrix for the cost of storing the full latent-state trajectory.

Formally, this quantity is the sensitivity of the current hidden state to the recurrent parameters. For a single recurrent hidden state $\mathbf{x}^{(t)}$ with recurrent parameters θ_{rec} , we define the influence matrix $\mathbf{M}^{(t)}$ as

$$\mathbf{M}^{(t)} = \frac{\partial \mathbf{x}^{(t)}}{\partial \theta_{\text{rec}}}. \quad (9)$$

Applying the chain rule yields the standard RTRL recursion

$$\begin{aligned} \mathbf{M}^{(t)} &= \frac{\partial \mathbf{x}^{(t)}}{\partial \theta_{\text{rec}}} + \frac{\partial \mathbf{x}^{(t)}}{\partial \mathbf{x}^{(t-1)}} \frac{\partial \mathbf{x}^{(t-1)}}{\partial \theta_{\text{rec}}} \\ &= \underbrace{\bar{\mathbf{M}}^{(t)}}_{\text{immediate influence}} + \underbrace{\mathbf{J}^{(t)} \mathbf{M}^{(t-1)}}_{\text{historic influence}}, \end{aligned} \quad (10)$$

where $\bar{\mathbf{M}}^{(t)}$ is the immediate influence of θ_{rec} on the current hidden state and $\mathbf{J}^{(t)} = \partial \mathbf{x}^{(t)} / \partial \mathbf{x}^{(t-1)}$ is the Jacobian of the hidden-state transition. Once $\mathbf{M}^{(t)}$ is available, the exact gradient of any instantaneous loss $\mathcal{L}^{(t)}$ with respect to the recurrent parameters can be written as

$$\frac{\partial \mathcal{L}^{(t)}}{\partial \theta_{\text{rec}}} = \frac{\partial \mathcal{L}^{(t)}}{\partial \mathbf{x}^{(t)}} \mathbf{M}^{(t)}. \quad (11)$$

Thus, unlike one-step updates that only account for the current application of the recurrent parameters, RTRL accumulates their full recursive influence through the hidden-state trajectory. Note that

Eq. (11) yields the same gradient as BPTT, just computed differently.

The main drawback of RTRL relates to computational cost. For a dense recurrent layer with n hidden units and $P = O(n^2)$ recurrent parameters, the influence matrix has size $n \times P = O(n^3)$ per layer. In multilayer recurrent networks the situation is more severe as each hidden layer depends on the parameters of all recurrent layers below it, and a naive L -layer construction requires $L(L+1)/2$ influence matrices [45]. Exact RTRL is therefore rarely used in large dense architectures. Several works make RTRL tractable through low-rank, factorised or sparse approximations to the influence matrix, trading exactness of $M^{(t)}$ for reduced computational cost and smaller memory requirements [46–48]. Another approach is to use element-wise recurrent models, where each hidden unit depends only on its own recurrent parameters [45, 49]. In this setting, the influence matrix becomes block-sparse and $M^{(t)}$ can be maintained without needing any approximation, whilst model performance often remains competitive with dense counterparts.

3. Temporal Predictive Coding with Real-Time Recurrent Learning

The previous section highlighted a structural limitation of tPC where the recurrent parameter update captures only the immediate application of the recurrent dynamics. In this section, we introduce

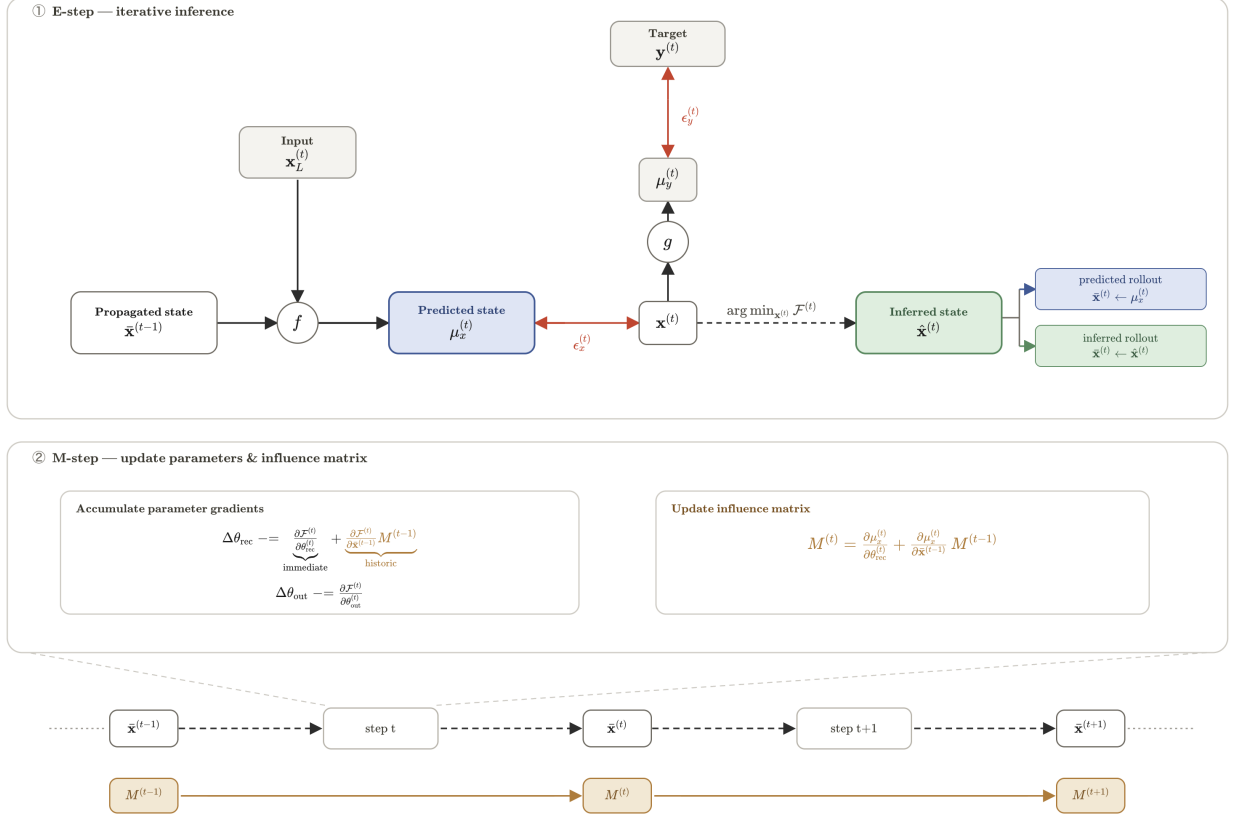


Figure 2: Overview of standard tPC and tPC-RTRL at a single timestep. The upper panel shows the shared tPC E-step. Here, the recurrent prior f produces a **predicted** latent state (blue) for the current timestep; iterative inference then produces the **inferred** state (green) by minimising local **prediction errors** (red). The middle panel shows the corresponding M-step. Standard tPC uses only the immediate component of the update, whereas tPC-RTRL additionally maintains an online **influence matrix** that includes its historic contribution to the current hidden state. **Gold** elements denote components introduced by tPC-RTRL. The lower panel illustrates the forward propagation of the latent state and influence matrix across timesteps.

Temporal Predictive Coding with Real-Time Recurrent Learning (tPC-RTRL), which augments tPC with an online influence matrix that aims to restore long-range temporal credit assignment whilst preserving the local structure of PC.

Figure 2 provides an overview of the method, contrasting it with tPC. The upper panel shows the standard tPC inference computation shared by tPC and tPC-RTRL. The recurrent prior predicts the current latent state, and iterative inference adjusts that state to reduce local prediction errors. The middle panel shows the difference in the recurrent M-step. Standard tPC uses only the immediate parameter contribution, whereas tPC-RTRL introduces an influence matrix to carry historic parameter effects forward through time. The lower panel shows

the two quantities propagated between successive timesteps.

3.1. Predicted, inferred, and propagated latent states

To read Figure 2, it is useful to distinguish three latent-state quantities with different computational roles. The state $\bar{\mathbf{x}}^{(t-1)}$ is the propagated latent state received from the previous timestep. Given the current input and this propagated state, the recurrent prior f produces the predicted latent state

$$\mu_x^{(t)} = f(\mathbf{x}_L^{(t)}, \bar{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}}). \quad (12)$$

The E-step then uses iterative inference to produce the inferred latent state $\hat{\mathbf{x}}^{(t)}$, which reduces the instantaneous free energy by balancing the recurrent prior prediction with the current output observation.

The associated prediction errors provide the local signals used for parameter updates.

The distinction between $\mu_x^{(t)}$, $\hat{\mathbf{x}}^{(t)}$, and $\bar{\mathbf{x}}^{(t)}$ is central to the method. The predicted state is the feedforward recurrent transition, the inferred state is its observation-corrected counterpart, and the propagated state determines which of these quantities becomes the input to the next recurrent transition. In Figure 2, blue denotes the predicted state, green denotes the inferred state and its optional propagation path, red denotes local prediction errors, and gold denotes the additional temporal-influence quantities introduced by tPC-RTRL.

A practically important design choice is therefore whether the propagated state is the inferred state, $\bar{\mathbf{x}}^{(t)} = \hat{\mathbf{x}}^{(t)}$, or the recurrent prediction, $\bar{\mathbf{x}}^{(t)} = \mu_x^{(t)}$. The two alternatives are shown at the right of the upper panel in Figure 2. The first option, choosing the inferred state and setting $\bar{\mathbf{x}}^{(t)} = \hat{\mathbf{x}}^{(t)}$, ensures that each recurrent transition is conditioned on the latest latent correction. This may be natural in filtering-style settings, where new observations are available online at test-time and could be used to refine the internal state. However, when training autoregressive sequence models, it may not make sense to propagate $\hat{\mathbf{x}}^{(t)}$. By propagating this corrected latent state when training, we are conditioning future dynamics on corrected states rather than the model’s own autonomous predictions. In this regime, inference can continually repair the latent trajectory, reducing pressure on the recurrent dynamics to learn good open-loop rollouts, leading to poor performance when corrections are not available at test-time.

In the settings considered in the main text, predictive performance is the primary objective. We therefore use the predictive-rollout choice

$$\bar{\mathbf{x}}^{(t)} = \mu_x^{(t)}, \quad (13)$$

and use the inferred state only to generate local prediction errors and parameter updates. This is the blue rollout branch in Figure 2, and it is the regime implemented by Algorithm 1. The inferred-propagation alternative is discussed in Appendix F.

3.2. tPC-RTRL

The middle and lower panels of Figure 2 show the additional computation introduced by tPC-RTRL. Standard tPC already provides the local inference dynamics in the upper panel, but its recurrent update accounts only for the immediate effect of θ_{rec}

on the current prediction. tPC-RTRL augments this update with an online influence matrix that stores how recurrent parameters have affected the latent state propagated through earlier timesteps.

To account for the missing temporal dependency in Eq. (8), we introduce an influence matrix

$$\mathbf{M}^{(t)} = \frac{\partial \bar{\mathbf{x}}^{(t)}}{\partial \theta_{\text{rec}}}, \quad (14)$$

which tracks how the recurrent parameters influence the propagated latent state. The recurrent parameter update can then be augmented to include both immediate and historic contributions:

$$\Delta \theta_{\text{rec}} = - \left(\underbrace{\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}^{(t)}}}_{\text{immediate}} + \underbrace{\frac{\partial \mathcal{F}^{(t)}}{\partial \bar{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)}}_{\text{historic}} \right). \quad (15)$$

This is the tPC analogue of the RTRL update rule. The immediate term captures the current use of the recurrent parameters, whilst the historic term propagates their effect forward through the latent-state trajectory. The propagation choice of Section 3.1 determines whether the recursion for $\mathbf{M}^{(t)}$ is exact or an approximation to the true influence of θ_{rec} on the current latent state. We discuss both regimes below.

Predictive-rollout regime (exact recursion). When $\bar{\mathbf{x}}^{(t)} = \mu_x^{(t)}$, the influence update is exact:

$$\mathbf{M}^{(t)} = \frac{\partial \mu_x^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} + \frac{\partial \mu_x^{(t)}}{\partial \bar{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)}. \quad (16)$$

In this regime, tPC-RTRL coincides with exact RTRL on the recurrent latent trajectory executed by the feedforward components of the model. All experiments in the main text of this paper use this regime.

Inferred-propagation regime (approximate recursion). When $\bar{\mathbf{x}}^{(t)} = \hat{\mathbf{x}}^{(t)}$, the transition from $\bar{\mathbf{x}}^{(t-1)}$ to $\bar{\mathbf{x}}^{(t)}$ is defined implicitly by iterative inference rather than a closed-form recurrence, and the exact influence recursion is no longer available. In this case there are two natural approximations. One can continue to update the influence matrix using Eq. (16), or alternatively evaluate the recursion using derivatives of the recurrent prediction at the converged inferred state:

$$\mathbf{M}^{(t)} \approx \left[\frac{\partial \mu_x^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} + \frac{\partial \mu_x^{(t)}}{\partial \bar{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)} \right]_{\mu_x^{(t)} = \hat{\mathbf{x}}^{(t)}}. \quad (17)$$

For some models, Eqs. (16) and (17) may be exactly the same. For example, if the model uses a linear projection as its transition function, these derivatives do not depend on the value of the current hidden state. A discussion of tPC-RTRL in this inferred-propagation regime is given in Appendix F and Section 5.6.

Importantly, the tPC-RTRL modifications affect only the recurrent learning rule. The inference dynamics remain exactly those of tPC, and non-recurrent parameters continue to use the standard tPC update, descending the gradient of the instantaneous free energy with respect to their current application, as this captures their full contribution:

$$\Delta\theta_{\text{out}} = -\frac{\partial\mathcal{F}^{(t)}}{\partial\theta_{\text{out}}^{(t)}}. \quad (18)$$

tPC-RTRL therefore preserves the local nature of PC, whilst augmenting the recurrent weight update with an online estimate of long-range temporal influence.

3.3. Algorithmic summary

The recurrent parameter update of Eq. (15) augments the standard tPC rule with an online estimate of long-range temporal influence, whilst leaving the inference dynamics and non-recurrent parameter updates unchanged. Throughout this work, we adopt the forward-update convention of Qi et al. [43] where parameter updates move the feedforward predictions towards the converged values. This choice keeps parameter updates anchored to the feedforward trajectory the model follows at test-time and has been shown to improve performance. Realising the forward-update convention requires a small amount of additional bookkeeping to implement, which we detail in Appendix B. We leave the notation in the main text unchanged to keep the presentation of the algorithm as clear as possible and in keeping with standard PC and tPC definitions.

In practice, the recurrent gradients in Eq. (15) may be applied online at each timestep, or accumulated across a sequence and applied in a time-batched manner. We adopt the latter, which avoids issues with stale gradients and is more convenient in minibatch training.

Algorithm 1 is the pseudocode counterpart of the predictive-rollout branch in Figure 2. At each timestep, the recurrent prior first computes the predicted state $\mu_x^{(t)}$. Iterative inference then produces

$\hat{\mathbf{x}}^{(t)}$ and the local prediction errors required for learning, without replacing the state propagated by the recurrent dynamics. In parallel, the influence matrix $\mathbf{M}^{(t)}$ is updated forward in time. The recurrent gradient accumulator receives both the immediate contribution from the current transition and the historic contribution carried by $\mathbf{M}^{(t-1)}$, while the predicted state $\mu_x^{(t)}$ is propagated to the next timestep.

3.4. Relation to Backpropagation-Through-Time

PC’s relationship to BP has been a central thread in the recent PC literature. A series of results have shown that PC can approximate BP gradients in static feedforward networks [19, 20] and, under explicit assumptions, recover them exactly across general computation graphs [21, 22]. Beyond helping to clarify what PC may be doing as an optimisation algorithm, these equivalences demonstrate that the spatial locality of PC does not come at the cost of the gradient quality that has made BP so successful.

Here we extend these proofs to tPC-RTRL. In the predictive-rollout regime introduced in Section 3.2, the RTRL recursion is exact and the tPC-RTRL recurrent update recovers BPTT gradients under explicit assumptions on inference and the readout pathway. This shows that the locality properties demanded by neuromorphic hardware (Section 2.1) can be satisfied in both space and time without sacrificing the optimisation quality of BPTT.

Proposition 1 (tPC-RTRL recovers BPTT in the predictive-rollout regime). *Assume (i) inference converges at each timestep, (ii) the fixed-prediction assumption [20] holds for the readout network, and (iii) the propagated state follows the predictive-rollout $\bar{\mathbf{x}}^{(t)} = \mu_x^{(t)}$. Then the tPC-RTRL recurrent updates, summed over a sequence of length T , recover the exact BPTT gradient with respect to the recurrent parameters θ_{rec} :*

$$\sum_{t=1}^T \left[\frac{\partial\mathcal{F}^{(t)}}{\partial\theta_{\text{rec}}^{(t)}} + \frac{\partial\mathcal{F}^{(t)}}{\partial\bar{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)} \right] = \frac{\partial}{\partial\theta_{\text{rec}}} \sum_{t=1}^T \ell_t, \quad (19)$$

where $\mathbf{M}^{(t)} = \partial\bar{\mathbf{x}}^{(t)}/\partial\theta_{\text{rec}}$ is the RTRL influence matrix.

The argument combines two points. A spatial PC equivalence at the recurrent state gives $\epsilon_x^{(t)} = -\partial\ell_t/\partial\bar{\mathbf{x}}^{(t)}$ at inference equilibrium, so the PC error acts as the backpropagated signal through the non-recurrent layers. The exact RTRL recursion on the propagated trajectory (Eq. 16) then provides

Algorithm 1 One training step of tPC-RTRL in the predictive-rollout regime (the blue rollout branch in Figure 2).

Require: Sequence $(\mathbf{x}_L^{(1)}, \dots, \mathbf{x}_L^{(T)})$ with targets $(\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(T)})$; parameters $\theta_{\text{rec}}, \theta_{\text{out}}$; learning rate η .

- 1: Initialise hidden state $\bar{\mathbf{x}}^{(0)}$; set influence matrix $\mathbf{M}^{(0)} \leftarrow \partial \bar{\mathbf{x}}^{(0)} / \partial \theta_{\text{rec}}$.
- 2: Initialise gradient accumulators $\Delta \theta_{\text{rec}} \leftarrow 0$, $\Delta \theta_{\text{out}} \leftarrow 0$.
- 3: **for** $t = 1, \dots, T$ **do**
- 4: $\mu_x^{(t)} \leftarrow f(\mathbf{x}_L^{(t)}, \bar{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}})$ ▷ predict
- 5: Run iterative inference to obtain $\hat{\mathbf{x}}^{(t)}$ minimising $\mathcal{F}^{(t)}$ ▷ E-step
- 6: $\mathbf{M}^{(t)} \leftarrow \partial \mu_x^{(t)} / \partial \theta_{\text{rec}}^{(t)} + (\partial \mu_x^{(t)} / \partial \bar{\mathbf{x}}^{(t-1)}) \mathbf{M}^{(t-1)}$ ▷ exact recursion
- 7: Accumulate recurrent grad: $\Delta \theta_{\text{rec}} \leftarrow \Delta \theta_{\text{rec}} + \partial \mathcal{F}^{(t)} / \partial \theta_{\text{rec}}^{(t)} + (\partial \mathcal{F}^{(t)} / \partial \bar{\mathbf{x}}^{(t-1)}) \mathbf{M}^{(t-1)}$ ▷ M-step (recurrent)
- 8: Accumulate readout grad: $\Delta \theta_{\text{out}} \leftarrow \Delta \theta_{\text{out}} + \partial \mathcal{F}^{(t)} / \partial \theta_{\text{out}}$ ▷ M-step (readout)
- 9: $\bar{\mathbf{x}}^{(t)} \leftarrow \mu_x^{(t)}$ ▷ predictive-rollout
- 10: **end for**
- 11: $\theta_{\text{rec}} \leftarrow \theta_{\text{rec}} + \eta \Delta \theta_{\text{rec}}$; $\theta_{\text{out}} \leftarrow \theta_{\text{out}} + \eta \Delta \theta_{\text{out}}$

the historic terms of the tPC-RTRL update. Summing over time gives the BPTT gradient. The full derivation is in Appendix D.

Scope of the equivalence and our experimental regime. Proposition 1 identifies an operating point at which tPC-RTRL coincides exactly with BPTT. Its role in this paper is to establish that the BPTT gradient lies within the family of updates tPC-RTRL can produce.

However, our experiments deliberately do not enforce this regime. We do this for a few reasons. First, guaranteeing convergence at each timestep, at least to the precision required for the theoretical BPTT equivalence, is likely to be difficult in practice, and would incur a computational cost that is ideally avoided. Furthermore, the fixed-prediction assumption [20] requires us to alter the inference dynamics, evaluating all derivatives during inference at their feedforward values. This would mean that inference is no longer performing gradient descent on the defined free energy, but instead on a slightly modified objective, and would, more fundamentally, mean that the inferred latent state no longer corresponds to an approximation of the model’s posterior over the latent. Under the variational interpretation of PC, the E-step performs approximate Bayesian inference of the latent given the current observation and previous inferred state [15, 24]. The filtering procedure of Section 4.5 exploits this property directly, clamping true-state observations at deployment time and allowing iterative inference to revise the latent trajectory in a way that is consistent with the generative model. Inference dynamics tuned to recover BPTT gradients would no longer necessarily

admit this interpretation, and might not allow for online state correction at test-time.

Furthermore, a growing body of work suggests that PC’s behaviour outside the equivalence regime is itself a source of its value, with reports of improved sample efficiency, robustness, and updates that capture aspects of higher-order optimisation [8, 23]. The empirical question we therefore ask in Section 4 is not whether tPC-RTRL can exactly match BPTT under tightly controlled assumptions, but whether, when allowed to operate outside of this controlled regime, it produces recurrent updates of comparable optimisation quality. We return to this question in Section 5.3.

3.5. Instantiation on the RG-LRU

The experiments in Sections 4.2–4.4 apply tPC-RTRL to a Real-Gated Linear Recurrent Unit (RG-LRU) [50]. We use this architecture because its recurrence is element-wise in the hidden dimension, which makes the exact RTRL recursion of Eq. (16) tractable. Each hidden unit depends only on its own row of parameters, so no cross-unit influences need to be tracked. Here we provide the cell definition, the per-timestep free energy used during inference, the element-wise RTRL recursion, and the resulting recurrent gradient. Further information including the regression-output variant used on the nanodrone model, and the optional input-projection and learnt state-initialisation heads is deferred to Appendix C.

For clarity, the tPC and tPC-RTRL formulations of Sections 2.3–3.2 were stated with a single readout layer, so that the recurrent latent $\mathbf{x}^{(t)}$ directly predicts the output by $g(\cdot; \theta_{\text{out}})$. The RG-LRU models

we use in the presented experiments have multiple readout layers. In this setting, our free energy formulation changes to include a latent state at each layer. For example, Eq. (25) minimises the free energy jointly over the recurrent latent $\mathbf{x}^{(t)}$ and an intermediary readout latent $\mathbf{o}^{(t)}$. This is a straightforward extension of the single-layer case, where we are effectively doing tPC over the recurrent latents, and PC over the readout latents as they lack the recurrent prior. This means that all layers, both recurrent and non-recurrent, are updated according to layer-local rules, regardless of the depth of the readout stack.

RG-LRU transition. Writing $\mathbf{x}_L^{(t)} \in \mathbb{R}^I$ for the input to the cell and $\bar{\mathbf{x}}^{(t-1)} \in \mathbb{R}^H$ for the propagated recurrent state from the previous timestep, the recurrent prediction $\mu_x^{(t)} \in \mathbb{R}^H$ is given by

$$g_a^{(t)} = \sigma(W_a \mathbf{x}_L^{(t)} + b_a), \quad (20)$$

$$g_z^{(t)} = \sigma(W_z \mathbf{x}_L^{(t)} + b_z), \quad (21)$$

$$\log a^{(t)} = c g_a^{(t)} \odot \log \sigma(\Lambda), \quad (22)$$

$$\gamma^{(t)} = \sqrt{1 - \exp(2 \log a^{(t)})}, \quad (23)$$

$$\mu_x^{(t)} = a^{(t)} \odot \bar{\mathbf{x}}^{(t-1)} + \gamma^{(t)} \odot (g_z^{(t)} \odot \mathbf{x}_L^{(t)}), \quad (24)$$

with recurrent parameters $\theta_{\text{rec}} = \{\Lambda, W_a, b_a, W_z, b_z\}$, a fixed scalar c , and $\Lambda \in \mathbb{R}^H$. The two gates $g_a^{(t)}, g_z^{(t)} \in (0, 1)^H$ modulate the decay $a^{(t)}$ and the input pathway respectively, and the recurrence is element-wise in the hidden dimension as each coordinate of $\mu_x^{(t)}$ depends only on the corresponding coordinate of $\bar{\mathbf{x}}^{(t-1)}$. When the input dimensionality differs from the hidden dimensionality, an additional linear projection $W_{\text{in}}, b_{\text{in}}$ is inserted in front of $\mathbf{x}_L^{(t)}$ in the $g_z^{(t)} \odot \mathbf{x}_L^{(t)}$ product; we keep this implicit in the main text and defer the details to Appendix C.

Free energy and inference. The RG-LRU is wrapped in a ReLU readout layer and a linear output head. During inference, let $\mathbf{x}^{(t)}$ denote the inferred recurrent latent, $\mathbf{o}^{(t)}$ the inferred readout latent, $\mathbf{y}^{(t)}$ the target, and ϕ the ReLU. With Gaussian conditionals at the recurrent and readout states and a softmax conditional at the output [51], the per-timestep free

energy is

$$\begin{aligned} \mathcal{F}^{(t)} = & \frac{1}{2H} \|\mathbf{x}^{(t)} - \mu_x^{(t)}\|_2^2 \\ & + \frac{1}{2R} \|\mathbf{o}^{(t)} - \phi(W_r \mathbf{x}^{(t)} + b_r)\|_2^2 \\ & + \frac{1}{C} \ell_{\text{CE}}(W_l \mathbf{o}^{(t)} + b_l, \mathbf{y}^{(t)}), \end{aligned} \quad (25)$$

where W_r, b_r are the readout weights, W_l, b_l are the output-head weights, R is the readout dimension, and C is the output dimension. This gives readout parameters $\theta_{\text{out}} = \{W_r, b_r, W_l, b_l\}$. The per-layer normalisations $1/H, 1/R, 1/C$ aim to keep the scale of each energy term independent of layer width. Inference minimises $\mathcal{F}^{(t)}$ jointly over $\{\mathbf{x}^{(t)}, \mathbf{o}^{(t)}\}$ via a fixed number of steps of momentum-accelerated gradient descent initialised at the feedforward values $\mathbf{x}^{(t)} \leftarrow \mu_x^{(t)}, \mathbf{o}^{(t)} \leftarrow \phi(W_r \mu_x^{(t)} + b_r)$. With the converged latent, $\hat{\mathbf{x}}^{(t)}$, the associated recurrent prediction error is given by

$$\hat{\epsilon}_x^{(t)} = \hat{\mathbf{x}}^{(t)} - \mu_x^{(t)} \quad (26)$$

and supplies the local signal used by the recurrent update below. The explicit gradient forms for $\partial \mathcal{F}^{(t)} / \partial \mathbf{x}^{(t)}$ and $\partial \mathcal{F}^{(t)} / \partial \mathbf{o}^{(t)}$, together with the inference step size and number of iterations used in each experiment, are given in Appendix C.1.

Element-wise RTRL recursion. Because the recurrent prediction in Eq. (24) is element-wise in the hidden dimension, the Jacobian of $\mu_x^{(t)}$ with respect to the propagated state collapses to $\partial \mu_x^{(t)} / \partial \bar{\mathbf{x}}^{(t-1)} = \text{diag}(a^{(t)})$. For any recurrent parameter $\theta \in \theta_{\text{rec}}$, let $\mathbf{M}_\theta^{(t)} = \partial \bar{\mathbf{x}}^{(t)} / \partial \theta$, the exact recursion of Eq. (16) then becomes

$$\mathbf{M}_\theta^{(t)} = a^{(t)} \odot \mathbf{M}_\theta^{(t-1)} + \frac{\partial \mu_x^{(t)}}{\partial \theta^{(t)}}, \quad (27)$$

where the chain term has reduced to element-wise multiplication by $a^{(t)}$ along the hidden axis. Since each hidden unit depends only on its own row of θ_{rec} , the influence tensors carry only a single hidden dimension, rather than a full $H \times H$ Jacobian, and the total per-example recurrent storage is $O(H + HT)$, as summarised in Section 5.2.

Recurrent parameter update. In the predictive-rollout regime, the immediate and historic terms of Eq. (15) combine through the influence tensor of

Eq. (27), and the time-batched gradient accumulator $\Delta\theta_{\text{rec}}$ of Algorithm 1 takes the simple form

$$\Delta\theta = \frac{1}{H} \sum_{t=1}^T \hat{\epsilon}_x^{(t)} \cdot \mathbf{M}_\theta^{(t)} \quad \text{for each } \theta \in \theta_{\text{rec}}, \quad (28)$$

applied at the end of the sequence as $\theta \leftarrow \theta + \eta \Delta\theta$. The inference dynamics, the RTRL recursion, and the readout-parameter updates are all applied layer-locally to the cell, with no global BPTT-style reverse traversal through time. This is the form of the tPC-RTRL update used in all RG-LRU experiments reported in Section 4.

4. Results

Section 2.3 identified a structural limitation of standard tPC, showing that its recurrent parameter update neglects the historic influence of the recurrent parameters along the latent-state trajectory, restricting the model’s ability to assign credit across long temporal horizons. The experiments in this section are organised around this limitation. We begin with the copy task (Section 4.1), which serves as a controlled diagnostic in which the failure of standard tPC, and the recovery offered by tPC-RTRL, can be cleanly isolated. We then ask whether this advantage persists in more realistic sequence-modelling problems and larger recurrent models, evaluating tPC-RTRL on byte-level language modelling (Section 4.2), machine translation (Section 4.3), and nanodrone system identification (Section 4.4). Finally, we present evidence that the iterative inference dynamics of tPC-RTRL admit a natural filtering interpretation at test-time (Section 4.5).

4.1. Copy Task

The copy task provides a controlled diagnostic of long-range temporal credit assignment, and cleanly separates methods that can effectively propagate gradient information across a temporal gap from those that cannot. We therefore use it to isolate the limitations of standard tPC identified in Section 2.3, and to show that tPC-RTRL’s augmentation of the recurrent update with an influence matrix is sufficient to restore long-range temporal credit assignment and solve the task.

Task definition. Figure 3 illustrates the copy task construction. The model is presented with a sequence of digits and must output the same sequence

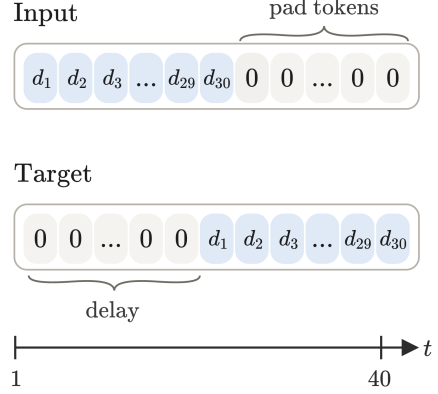


Figure 3: Copy task construction. A length-30 sequence of digits (d_i) is followed by padding in the input, while the target is the same sequence shifted into the future. d_i are sampled uniformly from the integers 1–9 and one-hot encoded.

after a fixed delay. For this task, the information required to predict each output is separated from the learning signal by several timesteps, making it difficult for models to learn without a mechanism for propagating gradient information across this gap. The input is a sequence of 30 digits sampled uniformly from the integers 1–9, with 0 reserved as a padding token. The target is the same sequence translated 10 timesteps into the future, with both sequences padded to a total of 40 timesteps.

Experimental setup. All models use a single-layer recurrent sequence model with 128 tanh hidden units and a per-timestep output head. Four models are trained on this task, one with each learning algorithm discussed in this work: tPC-RTRL, tPC, BPTT and spatial BP. Spatial BP represents a form of truncated BPTT where only local temporal information is used to calculate recurrent parameter gradients, detaching the hidden state at each timestep. All models are trained with a batch size of 16 and are evaluated on 200 randomly generated test samples every 16 batches. Reported values are means and standard deviations over 5 seeds, with the same seeds used for all methods. For full experimental details, see Appendix G.2.

Results. Table 1 shows a clear separation between methods that propagate recurrent credit through time and those that do not. BPTT and tPC-RTRL solve the task near-perfectly (accuracies of 99.99 ± 0.01 and 99.99 ± 0.02 respectively), achieving mean train and validation metrics matching to three significant figures. In contrast, spatial BP

and standard tPC converge to a substantially worse solution, with validation accuracy of around 40%. Overall, standard tPC closely matches spatial BP, whilst tPC-RTRL closely matches BPTT.

In this small and controlled setting, we are able to tune the PC inference dynamics so that tPC-RTRL reproduces BPTT almost exactly, and standard tPC reproduces spatial BP. These results support the claim that the failure mode of standard tPC on long-horizon sequence tasks is specifically a failure of recurrent credit assignment. On the copy task the relevant supervision arrives only after a long delay. A one-step recurrent update therefore appears insufficient to bridge the gap between presentation of the relevant information and the supervision signal, and the model fails to reliably learn. By contrast, tPC-RTRL overcomes this limitation by explicitly propagating recurrent influence through time, allowing tPC to recover the same optimisation behaviour as BPTT on this task.

4.2. Language Modelling on WikiText-103

The results of Section 4.1 establish that tPC-RTRL addresses the long-range credit-assignment limitation of tPC in a controlled setting. We now test whether this persists in more realistic sequence-modelling problems and larger recurrent models. Here, we study byte-level language modelling on WikiText-103 [28], and in Section 4.3, machine translation on a subset of CCMatrix [29]. In both experiments we use the Real-Gated Linear Recurrent Unit (RG-LRU) [50] introduced in Section 3.5. We choose this recurrent module because its element-wise recurrence makes exact RTRL-style influence tracking tractable at the widths used here, while still providing a richer gated recurrent architecture than the model used in the copy task.

Task definition. The corpus is converted directly to UTF-8 bytes and modelled as a sequence over a vocabulary of size 256. Training examples are formed by sampling, with replacement, random contiguous windows of length 257 from the training split. For each window, the first 256 bytes are used as inputs and the final 256 bytes are used as next-byte prediction targets. Validation and test evaluation use sequential windows constructed in the same way, slicing the whole dataset into input-output pairs without resampling.

Experimental setup. Four models are trained on this task, one with each learning algorithm. The

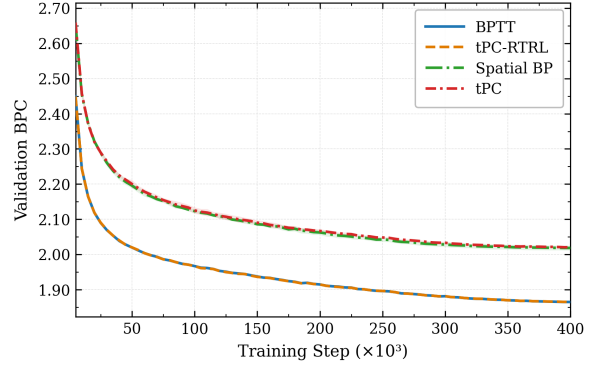


Figure 4: Validation bits-per-character on WikiText-103 as a function of training steps for BPTT, spatial BP, tPC, and tPC-RTRL. Lines show the mean over 5 seeds.

language model consists of a frozen pretrained byte embedding layer and the RG-LRU model introduced in Section 3.5. Each model has 1,313,536 total trainable parameters. Of these, 525,824 are recurrent parameters that require a temporal credit-assignment mechanism to ensure effective learning. Each model is trained for 400,000 training steps and evaluated every 5,000 steps on the validation set. The best checkpoint per seed is selected using validation bits-per-character (BPC) and used to report final test metrics. Reported values are means and standard deviations over 5 seeds. For full experimental details, including model layer sizes, see Appendix G.3.

Results. Table 2 and Figure 4 show that tPC-RTRL closes the gap to BPTT on byte-level language modelling, whilst standard tPC tracks the spatial BP baseline throughout training. BPTT reaches a mean validation BPC of 1.864, and tPC-RTRL reaches 1.865, a difference of 0.001 BPC that falls near seed variance. In contrast, tPC (2.020) and spatial BP (2.018) plateau above the BPTT/tPC-RTRL pair, and the two curves remain closely coupled across all 400,000 training steps (see Figure 4).

The correspondence observed on the copy task therefore persists at the scale of WikiText-103. The near-zero residual gap between tPC-RTRL and BPTT at this scale further suggests that, at least for this model size and dataset, any optimisation differences introduced by PC do not measurably degrade performance once temporal credit assignment is handled correctly.

Method	Train loss ($\downarrow$)	Val loss ($\downarrow$)	Val acc. (%) ($\uparrow$)
Spatial BP	1.658 ± 0.044	1.652 ± 0.047	39.76 ± 1.79
tPC	1.657 ± 0.044	1.652 ± 0.049	39.79 ± 1.86
BPTT	0.058 ± 0.007	0.055 ± 0.007	99.99 ± 0.01
tPC-RTRL	0.058 ± 0.007	0.055 ± 0.008	99.99 ± 0.02

Table 1: Copy task performance, reported as mean $\pm$ standard deviation over 5 seeds. tPC-RTRL matches BPTT, whilst standard tPC matches spatial BP. This shows that the decisive factor on this task is not PC itself, but whether recurrent credit is propagated across time.

Method	Best val. BPC ($\downarrow$)	Test BPC ($\downarrow$)
Spatial BP	2.018 ± 0.006	2.051 ± 0.007
tPC	2.020 ± 0.002	2.054 ± 0.004
BPTT	1.864 ± 0.002	1.895 ± 0.001
tPC-RTRL	1.865 ± 0.001	1.895 ± 0.002

Table 2: Byte-level language modelling results on WikiText-103. Values are reported as mean $\pm$ standard deviation over 5 random seeds. The best validation BPC per seed is used for model selection; test BPC is evaluated on the corresponding best checkpoint.

4.3. Machine Translation on CCMatrix (English–French)

Next, we look at machine translation, another sequence modelling problem which we consider a stronger test of temporal credit assignment. This is because the model must consume an entire source sentence before producing any target token, introducing a long-range dependency between the corresponding components of the source and target sentence, similar to that of the copy task in Figure 3. Algorithms that do not effectively propagate gradient information through time are therefore expected to struggle.

Task definition. CCMatrix [29] is a large-scale parallel corpus of 1.5 billion sentence pairs across 200 languages. As it has no canonical benchmark split and is far larger than our compute budget admits, we construct an English–French subset by sampling 600,000 sentence pairs, allocating 500,000 for training, 50,000 for validation, and 50,000 for testing. Sentences are filtered to a maximum combined (source + target) length of 128 tokens, lowercased, and tokenised with a Byte-Pair Encoding (BPE) tokeniser [52] trained jointly on source and target sentences from the training split, yielding a vocabulary of 10,000 tokens.

Experimental setup. Four models are trained on this task, one with each learning algorithm. The model architecture mirrors the byte-level language model of

Section 4.2, now operating over BPE tokens. It consists of a frozen pretrained token embedding layer, and the RG-LRU model introduced in Section 3.5. All models contain 13,399,824 trainable parameters, of which 2,100,224 are recurrent parameters.

From epoch 5 onward we probabilistically drop target input tokens, increasing the drop probability throughout training. This aims to discourage teacher-forcing dependence and encourage the recurrent state to carry longer-range predictive information. Due to compute constraints we train one model per method and use the final checkpoint to report test metrics. Full details are presented in Appendix G.4.

Results. Table 3 and Figure 5 show that the observations seen on the copy task and WikiText-103 extend to machine translation. tPC-RTRL closely matches BPTT in both test perplexity and BLEU, whilst standard tPC tracks spatial BP throughout training. Note that the spatial BP and tPC curves show an increase in validation loss from epoch 5 onwards, suggesting the naive credit assignment mechanisms struggle to maintain performance under the input token dropout schedule. Translation appears to be a stronger test of long-range credit assignment than next-byte language modelling as the model must associate target tokens with source-side context separated by the full encoder-side rollout. The persistent gap between BPTT/tPC-RTRL and tPC/spatial-BP confirms that this dependency requires an effective temporal credit assignment mechanism, and that tPC-RTRL achieves this whilst relying only on local computations.

4.4. System Identification with tPC-RTRL

The experiments in Sections 4.1–4.3 have so far considered temporal credit assignment in discrete-token sequence tasks. To assess whether the BPTT/tPC-RTRL equivalence extends to continuous problems with nonlinear dynamics, we evaluate

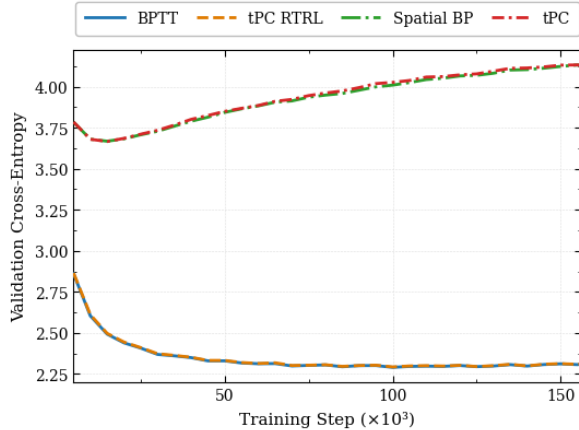


Figure 5: Validation cross-entropy loss (with label smoothing at 0.05) on the CCMatrix English–French translation task as a function of training steps for BPTT, spatial BP, tPC, and tPC-RTRL.

Method	Test PPL ($\downarrow$)	Test BLEU ($\uparrow$)
Spatial BP	24.56	4.58
tPC	24.53	4.75
BPTT	5.50	20.29
tPC-RTRL	5.52	20.23

Table 3: Perplexity (PPL) and BLEU score on the CCMatrix English–French subset. tPC-RTRL matches BPTT on both metrics, whilst standard tPC matches spatial BP. Metrics are a single run, with the final checkpoint used for evaluation.

the four learning algorithms on a nanodrone system-identification benchmark [30]. This section reports the nanodrone learning result; evidence for the filtering interpretation of tPC-RTRL, using the same dataset, is presented in Section 4.5.

Task definition. We use the Nonlinear System Identification Nanodrone Benchmark [30], a real-flight dataset collected on a Crazyflie 2.1 nano-quadrotor (mass ≈ 27 g, rotor span 92 mm). The benchmark comprises aggressive flight trajectories recorded at 100 Hz synchronising onboard motor commands with the drone’s position, velocity, orientation, and angular velocity measured by an external motion-capture system, or onboard gyroscope.

The goal is to take a 4-dimensional sequence of propeller angular velocities $u_t \in \mathbb{R}^4$ as input and predict the 9-dimensional drone state

$$s_t = [v_t^b, \phi_t, \omega_t] \in \mathbb{R}^9, \quad (29)$$

where $v_t^b \in \mathbb{R}^3$ is the linear velocity expressed in the

body frame, $\phi_t \in \mathbb{R}^3$ is the orientation encoded as a rotation vector, and $\omega_t \in \mathbb{R}^3$ is the angular velocity. Position is excluded from the direct prediction target and is recovered by integrating the predicted world-frame velocity, as is common in these types of task where position is typically unbounded.

Experimental setup. The dataset contains four trajectory types: **random**, **square**, **chirp**, and **melon**. We train on three families (**random**, **square**, **chirp**) using runs 1–3 of each trajectory type for training and run 4 for validation. The held-out **melon** trajectory (runs 1–3) is reserved entirely for testing, so generalisation is assessed on a trajectory type unseen during training.

Four models are trained on this task, one with each learning algorithm. All four share the single-layer RG-LRU cell of Section 3.5 with $H = 128$, with three changes appropriate to the nanodrone setting. These include the learnt input projection of Appendix C (since $I = 4 \neq H$), the tanh state-initialisation head of Eq. (C.3) in place of a zero initial state, and a mean squared error (MSE) output term (Eq. (C.4)) in place of softmax cross-entropy. The model contains 21,001 trainable parameters in total. Models are trained on rolling windows of $T = 200$ timesteps (2 seconds overall) with batch size 256 for 50 epochs. Reported values are means and standard deviations over 5 seeds. For full experimental details, see Appendix G.5.

Results. Table 4 reports rollout performance on the held-out **melon** trajectory. The equivalence observed in Sections 4.1–4.3 appears here again with tPC-RTRL closely matching BPTT across all three metrics, whereas standard tPC remains close to the spatial BP baseline. Specifically, tPC-RTRL achieves mean position, velocity, and attitude errors of 0.506 m, 0.808 m/s, and 0.232 rad, compared with 0.505 m, 0.809 m/s, and 0.230 rad for BPTT.

This result is important for two reasons. First, it shows that the BPTT/tPC-RTRL relationship is not confined to synthetic long-range benchmarks or discrete-token language tasks, but persists in a realistic continuous setting. Second, it further supports the diagnosis that the key limitation of standard tPC in recurrent settings is temporal credit assignment, and once recurrent influence is tracked through time, PC-based learning recovers BP-like performance even on this challenging system-identification problem.

Method	Mean pos. err. (m) ↓	Mean vel. err. (m/s) ↓	Mean att. err. (rad) ↓
Spatial BP	0.578 ± 0.013	0.924 ± 0.019	0.247 ± 0.015
tPC	0.556 ± 0.015	0.911 ± 0.020	0.253 ± 0.006
BPTT	0.505 ± 0.015	0.809 ± 0.037	0.230 ± 0.006
tPC-RTRL	0.506 ± 0.015	0.808 ± 0.027	0.232 ± 0.005

Table 4: Nanodrone rollout performance on the held-out **melon** trajectory. Values are mean rollout errors over the full 200-step horizon, reported as mean $\pm$ standard deviation over 5 seeds. Att. err. denotes rotation-vector error in radians.

4.5. Filtering with tPC-RTRL

A structural feature of tPC-based learning that distinguishes it from purely RTRL-based approaches is that the iterative inference mechanism used during training is also available at deployment time. In a filtering or state-estimation setting, observations of the true state may arrive periodically during deployment, and the model should incorporate these corrections into its internal trajectory. Under tPC-RTRL, the natural way to do this is to clamp the observation as a target in the free energy objective (Eq. 3) and run iterative inference, which revises the internal latent state without any modification to the learning algorithm.

This section presents evidence that this mechanism is useful in practice. The results below are encouraging in the high-correction-frequency regime but become marginal as the correction interval grows, so we identify several concrete directions for future work in Section 5.6.

Task Definition. We take the tPC-RTRL models from Section 4.4 and evaluate them on the held-out **melon** trajectory under a range of test-time correction schedules. At each correction step, the full ground-truth state is revealed. We compare three modes:

- *Pure rollout:* no correction, the model evolves autonomously for the full sequence.
- *Amortised:* the latent state is reset directly using the feedforward state-initialisation network.
- *tPC Inference:* the observation is clamped as the target and iterative free energy minimisation is used to update the latent state before rollout continues.

Between corrections, all methods evolve under the same learnt recurrent dynamics. We vary the correction period over $k \in \{10, 25, 50, 100\}$ steps, with an initial observation always taken at $t = 0$, and report mean $\pm$ standard deviation over 5 runs.

Results. Table 5 and Figure 6 summarise the outcome. Incorporating intermittent observations at test-time substantially improves over pure open-loop rollout. Table 5 shows that with no correction, the model reaches a final position error of 0.805 ± 0.034 m. When $k = 10$, tPC inference reduces this to 0.402 ± 0.035 m, compared with 0.560 ± 0.042 m for amortised correction. At the highest correction frequency, inference therefore cuts final position error by approximately half relative to pure rollout, and remains clearly better than a direct reset of the latent state.

The advantage of tPC inference is largest when observations are frequent and decreases as the correction interval grows. At $k = 25$, tPC inference outperforms amortised correction (0.504 vs. 0.607 m final position error) and yields lower rollout-averaged velocity and attitude errors. At $k = 50$, the remaining benefit is more modest, tPC inference still improves final position error and slightly improves velocity error, whilst attitude is essentially unchanged. By $k = 100$, the two correction schemes are very similar. tPC inference retains a small advantage in final position error (0.676 vs. 0.708 m) but the differences in rollout-averaged metrics are negligible, and inference is marginally worse than amortised correction in final attitude error (bottom right of Figure 6).

The fact that tPC inference is consistently better than the amortised reset at high correction frequencies, with no change to the learning algorithm or architecture, is encouraging evidence that the filtering interpretation of tPC-RTRL may have some practical utility. However, the narrowing advantage at longer correction intervals suggests further investigation is needed. We hypothesise two contributing causes. First, the nanodrone benchmark [30] has aggressive flight characteristics, and even with intermittent ground-truth corrections the drone state is difficult to track over relatively short timescales. This suggests that even a perfect correction mechanism is likely to degrade quickly. Second, the model has been trained in the predictive-rollout regime.

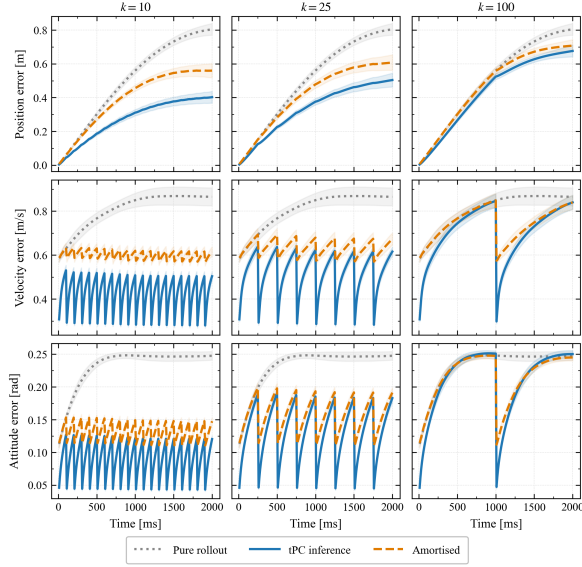


Figure 6: Rollout error on the held-out *melon* trajectory under intermittent test-time correction. Columns show correction periods of $k \in \{10, 25, 100\}$ steps; rows show position, velocity, and attitude error. Grey dotted lines: pure rollout. Dashed lines: amortised correction. Solid lines: tPC inference-based correction. Shaded regions indicate one standard deviation over 5 runs. $k = 50$ is omitted for brevity.

Therefore, during training, it experiences only regions of latent space reachable by the feedforward transition prior, so an inferred state applied at test-time may fall outside the region of the latent-state manifold visited during training. This means that the test-time distribution of latent states differs from that seen during training, and could possibly cause some degradation in subsequent rollout performance, providing a potential explanation for the degradation observed in the $k = 100$ attitude error (Figure 6, bottom right).

5. Discussion

5.1. Locality in space and time

The central conceptual contribution of this paper is that tPC-RTRL combines spatially local PC error signals with temporally local RTRL-style influence tracking.

Spatial locality is the standard appeal of PC where the global backward pass of BP is replaced by local prediction-error interactions at each layer. Each layer updates its parameters using only quantities available at its own inputs and outputs, without

requiring a globally coordinated reverse traversal of the forward graph. RTRL alone does not provide this. To obtain error signals at the recurrent state, a plain RTRL system must still backpropagate through the readout network, which requires the same phase-separated, globally coordinated backward pass that makes BPTT unattractive for neuromorphic hardware.

The temporal locality of RTRL is one of its most appealing properties. The full stored activation trajectory required by BPTT is replaced by an online influence matrix maintained forward in time. The memory cost becomes independent of sequence length, and no explicit unrolling is required. tPC alone does not provide this. We demonstrate in Sections 4.1–4.4, that standard tPC’s one-step recurrent update fails to propagate credit over long horizons and behaves like spatial BP.

Because the two mechanisms address orthogonal parts of the problem, their composition yields a credit-assignment mechanism that inherits the relevant locality properties of both components. Proposition 1, in Section 3.4, states the idealised consequence, under explicit assumptions, the composition recovers BPTT gradients exactly, and the experiments in Sections 4.1–4.4 provide empirical support that the algorithm survives relaxation of those assumptions in practice. A secondary structural benefit arises in filtering settings. As tPC already contains iterative inference as part of its learning machinery, the same mechanism can be reused at deployment time to incorporate online state corrections, as explored in Section 4.5. A pure RTRL system would require a separate mechanism for this, however the tPC-RTRL formulation readily permits this capability without further modification to the algorithm.

5.2. Memory implications

The temporal locality properties translate into a concrete memory implication (Figure 7). For the recurrent part of the nanodrone RG-LRU model used in Sections 4.4–4.5, BPTT retains the time-indexed activations needed to backpropagate through the recurrent trajectory. Excluding the readout pathway and focusing only on the state-transition part of the model, this consists of the input sequence $\mathbf{x}_L^{(1:T)} \in \mathbb{R}^{T \times I}$ and seven recurrent tensors in $\mathbb{R}^{T \times H}$: the projected input $p^{(t)}$, the two gates $g_z^{(t)}$ and $g_a^{(t)}$, the decay quantities $\log a^{(t)}$, $a^{(t)}$, and $\gamma^{(t)}$, and the

k	Method	Final pos. err. (m) ↓	Mean vel. err. (m/s) ↓	Mean att. err. (rad) ↓
—	Pure rollout	0.805 ± 0.034	0.808 ± 0.027	0.233 ± 0.005
10	Amortised	0.560 ± 0.042	0.601 ± 0.017	0.132 ± 0.005
	tPC Inference	0.402 ± 0.035	0.427 ± 0.019	0.089 ± 0.004
25	Amortised	0.607 ± 0.046	0.633 ± 0.022	0.158 ± 0.006
	tPC Inference	0.504 ± 0.041	0.519 ± 0.016	0.132 ± 0.004
50	Amortised	0.652 ± 0.042	0.678 ± 0.023	0.187 ± 0.006
	tPC Inference	0.586 ± 0.041	0.607 ± 0.016	0.174 ± 0.006
100	Amortised	0.708 ± 0.035	0.744 ± 0.021	0.216 ± 0.005
	tPC Inference	0.676 ± 0.035	0.705 ± 0.018	0.211 ± 0.006

Table 5: Test-time filtering performance on the held-out `melon` trajectory using the tPC-RTRL trained models. k denotes the number of rollout steps between corrections. Amortised correction resets the latent state using the feedforward state-initialisation network; tPC inference correction updates the latent state via iterative free energy minimisation. Reported as mean $\pm$ standard deviation over 5 runs.

recurrent state sequence $\bar{\mathbf{x}}^{(1:T)}$. This gives

$$TI + 7TH = 200 \cdot 4 + 7 \cdot 200 \cdot 128 = 180,000$$

stored values, or approximately 703 KB at float32. By contrast, tPC-RTRL, on the element-wise RG-LRU, stores only the current recurrent state $\bar{\mathbf{x}}^{(t)}$ together with the influence traces, $\mathbf{M}_\theta^{(t)}$, maintained online by the recurrent tracker. In this implementation those traces correspond to parameters Λ , W_a , b_a , W_z , b_z , W_{in} , b_{in} , W_{x_0} , and b_{x_0} , for a total of 3,328 influence values. Including the current 128-dimensional recurrent state gives 3,456 stored values in total, or approximately 13.5 KB, independent of sequence length.

At $T = 200$, this is about a $52\times$ reduction in the storage required for temporal credit assignment relative to BPTT. Furthermore, the BPTT requirement grows with T , at $T = 1,000$ it rises to approximately 3,516 KB, and at $T = 10,000$ to approximately 35,156 KB, whereas tPC-RTRL remains at 13.5 KB throughout. For streaming or long-horizon deployment scenarios, this sequence-length independence becomes significant.

This comparison concerns storage for recurrent credit assignment only; it does not include compute time, inference iterations, optimiser state, or non-recurrent activations.

5.3. Equivalence with BPTT

Proposition 1 establishes that tPC-RTRL coincides with BPTT exactly under the fixed-prediction assumption [20] and converged inference. As discussed in Section 3.4, our experiments do not enforce this regime, yet tPC-RTRL tracks BPTT closely

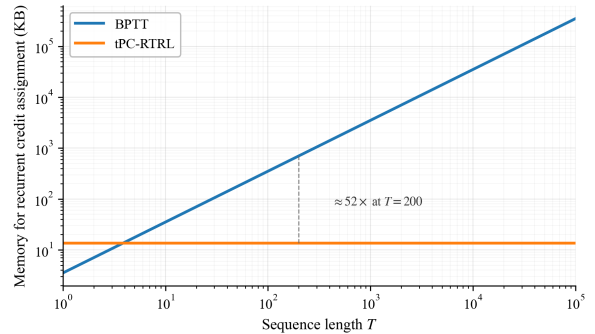


Figure 7: Memory scaling for recurrent credit assignment in the nanodrone RG-LRU model ($H = 128$, $I = 4$). BPTT memory grows as $TI + 7TH$, while tPC-RTRL remains constant at 3,456 stored values (approximately 13.5 KB). At $T = 200$, BPTT requires approximately 703 KB, giving a $52\times$ reduction for tPC-RTRL.

across four tasks. For each experiment, we hand-tuned the number of inference iterations and the inference momentum to maximise the cosine similarity between the tPC-RTRL and BPTT gradients on a single batch of untrained-model data. This served as a credible proxy for a full hyperparameter sweep over the inference dynamics, and was used to effectively select the inference parameters throughout Section 4. At these points, the cosine similarity between PC and BP gradient vectors on the recurrent parameters remains relatively close to 1 throughout training (see Appendix E for the measured similarity over training on WikiText-103), so the direction of the tPC-RTRL update is largely preserved relative to BPTT.

The residual differences appear mostly in magnitude rather than direction. Because at the start of inference, all the energy is concentrated at the network output, the per-layer norm of the PC gradient typically decreases from output to input, and the resulting tPC-RTRL update inherits a similar layer-wise magnitude profile. Adam’s per-parameter second-moment normalisation [53] absorbs such magnitude differences between the otherwise well-aligned gradient vectors. This combination, with high-cosine-similarity directions from the tuned inference dynamics, and adaptive per-parameter scaling from Adam, appears sufficient to minimise any resulting optimisation trajectory difference between BPTT and tPC-RTRL on the metrics tested.

What tPC-RTRL achieves in practice is therefore not strictly equivalent to BPTT. Proposition 1 establishes that the BPTT gradient lies within the family of updates tPC-RTRL can produce under tightly controlled assumptions. The empirical results then show that, at a tuned but practical operating point, the updates tPC-RTRL produces are aligned closely enough in direction, and Adam tolerant enough of the magnitude residuals, that BPTT-equivalent optimisation behaviour emerges without enforcing the proof’s assumptions. This framing is consistent with PC’s broader role in the literature, where its value in certain settings is reported to lie outside the strict-equivalence regime [8, 23]. Our results do not make any claim related to this, they simply report that, within the variance of our experiments, no measurable difference between BPTT and tPC-RTRL emerges on the tasks tested, and that the two methods track each other closely across training. Exploring conditions under which material differences between the two methods emerge, such as on significantly deeper models, and if tPC-RTRL carries the advantages of PC-based learning beyond the BPTT-equivalence regime, are identified as important directions for future work.

5.4. Scope of the hardware argument

We have motivated tPC-RTRL partly through its suitability for resource-constrained deployment. We frame this as a structural compatibility argument rather than an experimentally established systems advantage as we do not report energy measurements, FLOP costs, or deployment results on neuromorphic hardware. Translating the structural properties established above into measured energy or latency advantages on real hardware involves many additional factors, including memory bandwidth, inter-

core communication costs, and the overhead of the influence-matrix update itself. Characterising the full hardware profile of tPC-RTRL on a platform such as Loihi 2 [18] or SpiNNaker 2 [31] is a concrete and important direction for future work.

5.5. Architectural extensions

All experiments in this work use a single-layer RG-LRU recurrent module. This is a deliberate choice consistent with the edge-deployment target, but it is worth discussing how the approach extends to deeper recurrent stacks. For an L -layer stack, each hidden layer depends on the parameters of all layers below, and a naive extension of exact RTRL must maintain $L(L + 1)/2$ influence matrices [45]. Element-wise recurrent modules help limit the size of each individual influence matrix, but the inter-layer dependency structure means that higher layers cannot be computed from the local perspective of a single layer alone. Preserving the intra-layer locality that makes tPC-RTRL attractive for neuromorphic hardware therefore requires an approximation.

A promising route is the layer-local RTRL approximation of Zucchet et al. [49], in which each recurrent layer maintains its own influence matrix and cross-layer influence is ignored. The authors show that this approximation retains much of the expressivity gain of deeper networks in conventional recurrent architectures. This approximation composes naturally with tPC-RTRL as each layer still receives a local prediction-error signal from PC, and each layer’s parameter update remains computable from quantities local to that layer. A multi-layer tPC-RTRL system would therefore be local at the layer level in both the spatial and temporal axes, at the cost of exactness. Characterising the resulting gradient gap, and studying at what depths it becomes practically consequential, is a natural follow-up. Beyond the RG-LRU, tPC-RTRL could also be combined with other element-wise recurrent units [45, 54] or with sparse or factorised approximations of RTRL [55]. Progress on approximate or structured RTRL methods is therefore complementary to the tPC-RTRL framework proposed here.

5.6. Beyond predictive-rollout

We adopt the predictive-rollout regime throughout, propagating $\bar{\mathbf{x}}^{(t)} = \mu_x^{(t)}$ rather than the inferred state. This ensures consistency between training and test-time dynamics, incentivises the model to build good rollout performance rather than relying on

corrections, and keeps the RTRL recursion exact. However, during learning, iterative inference serves only as a means to obtain local parameter gradients, and does not revise the latent trajectory the model follows. The filtering results of Section 4.5 begin to suggest this may have implications. When an inferred-state correction is applied at test-time to a model trained under predictive-rollout, the corrected state may fall outside the region of latent space the learnt dynamics have experienced during training. In our filtering experiments this effect was small, visible only as a slight increase in final attitude error at the longest correction interval ($k = 100$, Table 5). However, it may present a more significant issue in other settings, perhaps in scenarios where open-loop rollout performance degrades less quickly, and intermittent corrections have a longer lasting influence on the trajectory. Characterising where the solutions found by iterative inference sit in the latent space relative to the feedforward rollout trajectory, to what degree the recurrent prior anchors the inferred state to familiar regions of this space, and quantifying the downstream consequences for rollout performance, are important directions for future work.

A related question is whether the training regime itself should adapt to inferred-state dynamics. Appendix F examines the inferred-propagation regime, in which $\hat{\mathbf{x}}^{(t)}$ is propagated throughout training under the approximate influence recursion of Eq. (17). On the copy task, the resulting gradient bias produces a modest degradation but still improves on standard tPC (Appendix F.1); on the nanodrone benchmark, training loss drops well below the predictive-rollout baseline whilst open-loop validation rollout substantially worsens. This is because inference corrections continuously repair the latent trajectory and the recurrent dynamics are no longer pressured to learn accurate autonomous rollouts (Appendix F.2). These small experiments suggest some areas for future work. First, quantifying how the approximation-induced gradient bias scales with distance between the inferred state and the predictive-rollout trajectory, and how this interacts with the training dynamics, is a necessary step toward understanding the conditions under which the induced bias is benign or harmful. Second, exploring hybrid regimes, analogous to scheduled sampling [56], that begin under predictive-rollout before gradually introducing inferred-state propagation, might allow the recurrent dynamics to first acquire the autonomous-rollout competence that predictive-rollout incen-

tivises, before then gradually exposing the model to the inferred-state distribution and encouraging it to learn dynamics that are robust to this shift. We did not pursue these further here because the filtering results of Section 4.5 suggest much of the practical benefit of iterative inference can be obtained at test-time without altering the training regime, so the immediate incentive to redesign training was limited; whether this changes for longer horizons or different rollout-degradation profiles remains open and we leave these as promising directions for future work.

6. Conclusion

This work introduced tPC-RTRL, an algorithm for recurrent neural networks that combines the local, parallelisable credit assignment of Temporal Predictive Coding (tPC) with an online influence matrix that tracks the recursive contribution of recurrent parameters across time. We showed that, under explicit assumptions, tPC-RTRL recovers the exact gradients computed by backpropagation-through-time (BPTT). Empirically, across four tasks including a controlled copy task, byte-level language modelling on WikiText-103, English–French translation on CCMatrix, and a realistic nanodrone system-identification benchmark, the observations are consistent. tPC-RTRL matches BPTT very closely, and standard tPC tracks one-step truncated BPTT. Filtering experiments on a nanodrone benchmark further show that the iterative inference mechanism used during training can also be used at test-time to incorporate intermittent state observations.

The broader conceptual claim is that Predictive Coding (PC) and Real-Time Recurrent Learning (RTRL) address complementary aspects of recurrent credit assignment. PC supplies local spatial error signals, while RTRL supplies online temporal influence tracking. Together, they yield a recurrent learning rule with locality properties that neither component provides in isolation. These properties align with the communication, memory, and synchronisation constraints that motivate neuromorphic and edge-hardware implementations. Their combination therefore extends tPC beyond one-step temporal credit assignment without discarding the inference-based structure that makes PC attractive.

This combination is relevant to continuously operating systems in which data arrive sequentially, memory is constrained, and intermittent observations may be available to correct an evolving inter-

nal state, including adaptive robotics, autonomous aerial vehicles, industrial monitoring, and wearable or edge-based sensing. In such settings, RTRL avoids BPTT's need to retain and reverse-traverse an unrolled state trajectory, and the PC component provides a mechanism for incorporating state observations at deployment time using the same iterative inference dynamics used during training. For the single-layer nanodrone model, this corresponds to an approximate $52\times$ reduction in the storage required specifically for temporal credit assignment relative to BPTT at a 200-step horizon, with this requirement independent of sequence length. More generally, tPC-RTRL suggests that long-range learning and observation-conditioned state correction can be brought together within a common framework.

Several directions for future work are open. First, extending tPC-RTRL to deeper recurrent stacks via the layer-local RTRL approximation of Zuchet et al. [49]. Second, a further investigation of the inferred-state propagation regime, including quantifying the approximation-induced gradient bias and exploring hybrid training regimes that gradually introduce inferred-state propagation. Third, whilst the structural arguments for compatibility with neuromorphic and edge hardware are concrete, demonstrating measured energy or latency advantages on real hardware, for example on Loihi 2 [18] or SpiNNaker 2 [31], remains essential future work. Addressing these questions represents the natural next step toward energy-efficient on-device training of recurrent sequence models.

7. Funding

This research was supported by the UK Government through the EPSRC Edgy Organism project (EP/Y030133/1).

References

- [1] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, *nature* 323 (1986) 533–536.
- [2] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, in: *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1, NIPS'12*, Curran Associates Inc., Red Hook, NY, USA, 2012, p. 1097–1105.
- [3] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, *CoRR* abs/1810.04805 (2018). URL: <http://arxiv.org/abs/1810.04805>. arXiv:1810.04805.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, 2013. URL: <https://arxiv.org/abs/1312.5602>. arXiv:1312.5602.
- [5] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, *CoRR* abs/1907.10597 (2019). URL: <http://arxiv.org/abs/1907.10597>. arXiv:1907.10597.
- [6] X. Xie, H. S. Seung, Equivalence of backpropagation and contrastive hebbian learning in a layered network, *Neural Computation* 15 (2003) 441–454. doi:10.1162/089976603762552988.
- [7] B. Scellier, Y. Bengio, Equilibrium propagation: Bridging the gap between energy-based models and backpropagation, 2017. doi:10.3389/fncom.2017.00024.
- [8] Y. Song, B. Millidge, T. Salvatori, T. Lukasiewicz, Z. Xu, R. Bogacz, Inferring neural activity before plasticity as a foundation for learning beyond backpropagation, *Nature Neuroscience* 27 (2024) 348–358. URL: <https://www.nature.com/articles/s41593-023-01514-1>. doi:10.1038/s41593-023-01514-1.
- [9] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain, Y. Liao, C.-K. Lin, A. Lines, R. Liu, D. Mathaikutty, S. McCoy, A. Paul, J. Tse, G. Venkataramanan, Y. Weng, A. Wild, Y. Yang, H. Yao, Loihi: A neuromorphic many-core processor with on-chip learning, *IEEE Micro* 38 (2018) 82–99.
- [10] R. P. Rao, D. H. Ballard, Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects, *Nature Neuroscience* 2 (1999) 79–87. doi:10.1038/4580.
- [11] D. Mumford, On the computational architecture of the neocortex. i. the role of the thalamo-cortical loop, *Biological Cybernetics* 65 (1991) 135–145. URL: <https://link.springer.com/>

- article/10.1007/BF00202389. doi:10.1007/BF00202389.
- [12] D. Mumford, On the computational architecture of the neocortex. ii. the role of cortico-cortical loops, *Biological Cybernetics* 66 (1992) 241–251. URL: <https://link.springer.com/article/10.1007/BF00198477>. doi:10.1007/BF00198477.
- [13] J. J. Atick, Could information theory provide an ecological theory of sensory processing?, *Network: Computation in Neural Systems* 22 (1991) 4–44. doi:10.3109/0954898X.2011.638888.
- [14] F. Attneave, Some informational aspects of visual perception, *Psychological Review* 61 (1954) 183–193. doi:10.1037/h0054663.
- [15] K. Friston, Learning and inference in the brain, *Neural Networks* 16 (2003) 1325–1352.
- [16] K. Friston, A theory of cortical responses, *Philosophical Transactions of the Royal Society B: Biological Sciences* 360 (2005) 815–836.
- [17] K. Friston, Hierarchical models in the brain, *PLoS Computational Biology* 4 (2008) e1000211.
- [18] G. Orchard, E. P. Frady, D. B. D. Rubin, S. Sanborn, S. B. Shrestha, F. T. Sommer, M. Davies, Efficient neuromorphic signal processing with loihi 2, *arXiv preprint arXiv:2111.03746* (2021). doi:10.48550/arXiv.2111.03746.
- [19] J. C. R. Whittington, R. Bogacz, An approximation of the error backpropagation algorithm in a predictive coding network with local hebbian synaptic plasticity, *Neural computation* 29 (2017) 1229 – 1262. URL: <https://api.semanticscholar.org/CorpusID:13651627>.
- [20] B. Millidge, A. Tschantz, C. L. Buckley, Predictive coding approximates backprop along arbitrary computation graphs, *Neural Computation* 34 (2022) 1329–1368. doi:10.1162/neco_a_01497.
- [21] Y. Song, T. Lukasiewicz, Z. Xu, R. Bogacz, Can the brain do backpropagation? — exact implementation of backpropagation in predictive coding networks, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, volume 33, Curran Associates, Inc., 2020, pp. 22566–22579.
- [22] T. Salvatori, Y. Song, T. Lukasiewicz, R. Bogacz, Z. Xu, Reverse differentiation via predictive coding, in: *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, 2022, pp. 8150–8158.
- [23] A. Mali, T. Salvatori, A. Ororbia, Tight stability, convergence, and robustness bounds for predictive coding networks, 2024. URL: <https://arxiv.org/abs/2410.04708>. arXiv:2410.04708.
- [24] B. Millidge, M. Tang, M. Osanlouy, N. S. Harper, R. Bogacz, Predictive coding networks for temporal prediction, *PLoS Computational Biology* 20 (2024) e1011183. doi:10.1371/journal.pcbi.1011183.
- [25] M. Tang, H. Barron, R. Bogacz, Sequential memory with temporal predictive coding, in: *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*, 2023.
- [26] L. P. Jiang, R. P. N. Rao, Dynamic predictive coding: A model of hierarchical sequence learning and prediction in the neocortex, *PLOS Computational Biology* 20 (2024) 1–30. URL: <https://doi.org/10.1371/journal.pcbi.1011801>. doi:10.1371/journal.pcbi.1011801.
- [27] A. Ororbia, A. Mali, C. L. Giles, D. Kifer, Continual learning of recurrent neural networks by locally aligning distributed representations, *IEEE Transactions on Neural Networks and Learning Systems* 31 (2020) 4267–4278. doi:10.1109/TNNLS.2019.2953622.
- [28] S. Merity, C. Xiong, J. Bradbury, R. Socher, Pointer sentinel mixture models, in: *International Conference on Learning Representations (ICLR 2017) Workshops*, 2017. Spotlight paper at the Workshop on Multi-class and Multi-label Learning in Extremely Large Label Spaces.
- [29] H. Schwenk, G. Wenzek, S. Edunov, E. Grave, A. Joulin, CCmatrix: Mining billions of high-quality parallel sentences on the web, in: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the*

- 11th International Joint Conference on Natural Language Processing, Association for Computational Linguistics, Online, 2021, pp. 6490–6500. doi:10.18653/v1/2021.acl-long.507.
- [30] R. Busetto, E. Cereda, M. Forgione, G. Maroni, D. Piga, D. Palossi, Nonlinear system identification for a nano-drone benchmark, *Control Engineering Practice* 172 (2026) 106871. URL: <http://dx.doi.org/10.1016/j.conengprac.2026.106871>. doi:10.1016/j.conengprac.2026.106871.
 - [31] H. A. Gonzalez, J. Huang, F. Kelber, et al., Spinnaker2: A large-scale neuromorphic system for event-based and asynchronous machine learning, *arXiv preprint arXiv:2401.04491* (2024). doi:10.48550/arXiv.2401.04491.
 - [32] F. Akopyan, J. Sawada, A. Cassidy, et al., Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34 (2015) 1537–1557. doi:10.1109/TCAD.2015.2474396.
 - [33] S. Schmitt, J. Klahn, G. Bellec, A. Grubl, M. Guttler, A. Hartel, S. Hartmann, D. Husmann, K. Husmann, S. Jeltsch, V. Karasenko, M. Kleider, C. Koke, A. Kononov, C. Mauch, E. Muller, P. Muller, J. Partzsch, M. A. Petrovici, S. Schiefer, S. Scholze, V. Thanasoulis, B. Vogginger, R. Legenstein, W. Maass, C. Mayr, R. Schuffny, J. Schemmel, K. Meier, Neuromorphic hardware in the loop: Training a deep spiking network on the brainscales wafer-scale system, in: 2017 International Joint Conference on Neural Networks (IJCNN), IEEE, 2017, p. 2227–2234. URL: <http://dx.doi.org/10.1109/IJCNN.2017.7966125>. doi:10.1109/ijcnn.2017.7966125.
 - [34] T. Wunderlich, A. F. Kungl, E. Müller, A. Hartel, Y. Stradmann, S. A. Aamir, A. Gröbl, A. Heimbrecht, K. Schreiber, D. Stöckel, C. Pehle, S. Billaudelle, G. Kiene, C. Mauch, J. Schemmel, K. Meier, M. A. Petrovici, Demonstrating advantages of neuromorphic computation: A pilot study, *Frontiers in Neuroscience* 13 (2019) 260. doi:10.3389/fnins.2019.00260.
 - [35] A. Rostami, B. Vogginger, Y. Yan, C. G. Mayr, E-prop on SpiNNaker 2: Exploring online learning in spiking RNNs on neuromorphic hardware, *Frontiers in Neuroscience* 16 (2022) 1018006. doi:10.3389/fnins.2022.1018006.
 - [36] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, W. Maass, A solution to the learning dilemma for recurrent networks of spiking neurons, *Nature Communications* 11 (2020) 3625. URL: <https://doi.org/10.1038/s41467-020-17236-y>. doi:10.1038/s41467-020-17236-y.
 - [37] A. Renner, F. Sheldon, A. Zlotnik, L. Tao, A. Sornborger, The backpropagation algorithm implemented on spiking neuromorphic hardware, *Nature Communications* 15 (2024). URL: <http://dx.doi.org/10.1038/s41467-024-53827-9>. doi:10.1038/s41467-024-53827-9.
 - [38] B. Millidge, A. K. Seth, C. L. Buckley, Predictive coding: a theoretical and experimental review, *CoRR abs/2107.12979* (2021). URL: <https://arxiv.org/abs/2107.12979>. doi:10.1038/s41467-024-53827-9.
 - [39] T. Salvatori, L. Pinchetti, B. Millidge, Y. Song, T. Bao, R. Bogacz, T. Lukasiewicz, Learning on arbitrary graph topologies via predictive coding, 2022. URL: <https://arxiv.org/abs/2201.13180>. doi:10.1038/s41467-024-53827-9.
 - [40] L. Pinchetti, C. Qi, O. Lokshyn, G. Olivers, C. Emde, M. Tang, A. M’Charrak, S. Frieder, B. Menzat, R. Bogacz, T. Lukasiewicz, T. Salvatori, Benchmarking predictive coding networks – made simple, 2024. URL: <https://arxiv.org/abs/2407.01163>. doi:10.1038/s41467-024-53827-9.
 - [41] N. Alonso, J. Krichmar, E. Neftci, Understanding and improving optimization in predictive coding networks, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024, pp. 10812–10820. doi:10.1609/aaai.v38i10.28954.
 - [42] S. Frieder, L. Pinchetti, T. Lukasiewicz, Bad minima of predictive coding energy functions, in: *The Second Tiny Papers Track at ICLR 2024*, 2024. URL: <https://openreview.net/forum?id=5Vc1ACZ8eC>.
 - [43] C. Qi, T. Lukasiewicz, T. Salvatori, Training deep predictive coding networks, in: *New Frontiers in Associative Memories*, 2025.

- URL: <https://openreview.net/forum?id=s3E08R4AMK>.
- [44] R. J. Williams, D. Zipser, A learning algorithm for continually running fully recurrent neural networks, *Neural Computation* 1 (1989) 270–280. doi:10.1162/neco.1989.1.2.270.
 - [45] K. Irie, A. Gopalakrishnan, J. Schmidhuber, Exploring the promise and limits of real-time recurrent learning, in: *International Conference on Learning Representations (ICLR)*, Vienna, Austria, 2024.
 - [46] A. Mujika, F. Meier, A. Steger, Approximating real-time recurrent learning with random kronecker factors, in: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018, pp. 6594–6603.
 - [47] C. Tallec, Y. Ollivier, Unbiased online recurrent optimization, in: *International Conference on Learning Representations (ICLR)*, 2018.
 - [48] J. Menick, E. Elsen, U. Evci, S. Osindero, K. Simonyan, A. Graves, Practical real time recurrent learning with a sparse approximation, in: *International Conference on Learning Representations (ICLR)*, 2021. URL: <https://openreview.net/forum?id=q3KSThy2GwB>, spotlight paper.
 - [49] N. Zucchet, R. Meier, S. Schug, A. Mujika, J. Sacramento, Online learning of long-range dependencies, in: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
 - [50] S. De, S. L. Smith, A. Fernando, A. Botev, G. Cristian-Muraru, A. Gu, R. Haroun, L. Berrada, Y. Chen, S. Srinivasan, G. Desjardins, A. Doucet, D. Budden, Y. W. Teh, R. Pascanu, N. D. Freitas, C. Gulcehre, Griffin: Mixing gated linear recurrences with local attention for efficient language models, 2024. URL: <https://arxiv.org/abs/2402.19427>. arXiv:2402.19427.
 - [51] L. Pinchetti, T. Salvatori, Y. Yordanov, B. Milledge, Y. Song, T. Lukasiewicz, Predictive coding beyond gaussian distributions, in: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, New Orleans, LA, USA, 2022, pp. 1280–1293.
 - [52] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, in: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2016, pp. 1715–1725. doi:10.18653/v1/P16-1162.
 - [53] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: *International Conference on Learning Representations (ICLR)*, 2015. URL: <https://iclr.cc/archive/www/doku.php?id=iclr2015:main>.
 - [54] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, Çağlar Gülçehre, R. Pascanu, S. De, Resurrecting recurrent neural networks for long sequences, in: *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 2023, pp. 26670–26698.
 - [55] O. Marschall, K. Cho, C. Savin, A unified framework of online learning algorithms for training recurrent neural networks, 2019. URL: <https://arxiv.org/abs/1907.02649>. arXiv:1907.02649.
 - [56] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, Scheduled sampling for sequence prediction with recurrent neural networks, 2015. URL: <https://arxiv.org/abs/1506.03099>. arXiv:1506.03099.

Appendix

This Appendix provides additional technical detail supporting the methods and empirical results presented in the main paper. The main text is intended to be self-contained; the material collected here is provided to support reproducibility and to clarify the derivations, implementation choices, and experimental procedures underlying Temporal Predictive Coding with Real-Time Recurrent Learning (tPC-RTRL).

Appendix A. Derivation of the PC free energy and local update rules

This appendix gives the full derivation summarised in Section 2.2 including the variational free energy formulation of PC, its reduction to a sum of local squared prediction errors under Gaussian conditionals and the mean-field approximation, and the resulting layer-local inference and learning rules.

PC can be viewed as performing approximate Bayesian inference in a hierarchical generative model. We consider a discriminative setting, in which the input is treated as the top-level latent variable, $\mathbf{x}_L$, whilst the observed target is denoted $\mathbf{y}$. The remaining variables $\mathbf{x}_{1:L-1}$ are latent and must be inferred. Writing the conditional generative model as

$$p(\mathbf{y}, \mathbf{x}_{1:L-1} \mid \mathbf{x}_L; \theta), \quad (\text{A.1})$$

exact inference would require the posterior $p(\mathbf{x}_{1:L-1} \mid \mathbf{y}, \mathbf{x}_L; \theta)$, which is generally intractable. PC introduces a variational posterior $q(\mathbf{x}_{1:L-1})$ and optimises the variational free energy

$$\mathcal{F}[q, \theta] = \mathbb{E}_q [\log q - \log p(\mathbf{y}, \mathbf{x}_{1:L-1} \mid \mathbf{x}_L; \theta)]. \quad (\text{A.2})$$

Equivalently, $\mathcal{F}$ is the negative evidence lower bound

$$\mathcal{F}[q, \theta] = \text{KL}[q \parallel p(\mathbf{x}_{1:L-1} \mid \mathbf{y}, \mathbf{x}_L; \theta)] - \log p(\mathbf{y} \mid \mathbf{x}_L; \theta). \quad (\text{A.3})$$

Minimising $\mathcal{F}$ with respect to q is therefore equivalent to minimising the Kullback–Leibler (KL) divergence to the exact posterior, since the evidence term is constant with respect to q .

To obtain the standard PC objective, we specify a layer-wise factorisation of the generative model with Gaussian conditionals,

$$p(\mathbf{x}_l \mid \mathbf{x}_{l+1}; \theta_l) = \mathcal{N}(\mathbf{x}_l; g_l(\mathbf{x}_{l+1}; \theta_l), \Sigma_l), \quad (\text{A.4})$$

for $l = 0, \dots, L-1$, with $\mathbf{x}_0 \equiv \mathbf{y}$. Assuming that q factorises under a mean-field approximation as

$$q(\mathbf{x}_{1:L-1}) = \prod_{l=1}^{L-1} q_l(\mathbf{x}_l), \quad (\text{A.5})$$

and that each factor is modelled either as a Gaussian with sufficiently small variance or as a Dirac delta centred on the current state estimate, $\mathcal{F}$ reduces to a sum of precision-weighted local prediction errors:

$$\mathcal{F} = \frac{1}{2} \sum_{l=0}^{L-1} \epsilon_l^\top \Pi_l \epsilon_l, \quad (\text{A.6})$$

$$\epsilon_l = \mathbf{x}_l - g_l(\mathbf{x}_{l+1}; \theta_l), \quad (\text{A.7})$$

with $\Pi_l = \Sigma_l^{-1}$. It is common for the precision terms to be fixed to the identity, in which case they can be omitted, and we do so throughout this work. The free energy objective therefore simplifies to

$$\mathcal{F} = \frac{1}{2} \sum_{l=0}^{L-1} \epsilon_l^\top \epsilon_l. \quad (\text{A.8})$$

PC is typically implemented using an Expectation-Maximisation (EM) procedure [19]. First, with weights fixed, the latent states $\mathbf{x}_{1:L-1}$ are iteratively updated to reduce $\mathcal{F}$ (an E-step). Then, with the states held fixed at their inferred values, the model parameters are updated by descending the same objective with respect to θ (an M-step). Inference and learning therefore correspond to alternating optimisation over variational parameters and model parameters of a single free energy functional.

Examining $\mathcal{F}$ makes clear why PC gives rise to layer-local rules for inference and learning. Inference corresponds to gradient descent on $\mathcal{F}$ with respect to the latent activities $\mathbf{x}_{1:L-1}$. For an internal layer $l \in \{1, \dots, L-1\}$,

$$\Delta \mathbf{x}_l = -\alpha \frac{\partial \mathcal{F}}{\partial \mathbf{x}_l} = \alpha \left(\left(\frac{\partial g_{l-1}(\mathbf{x}_l; \theta_{l-1})}{\partial \mathbf{x}_l} \right)^\top \epsilon_{l-1} - \epsilon_l \right), \quad (\text{A.9})$$

so the update to each state depends only on the prediction error at its own layer and the layer directly below. Similarly, each layer updates its weights using only locally available quantities:

$$\Delta W_l = -\eta \frac{\partial \mathcal{F}}{\partial \theta_l} = \eta \left(\frac{\partial g_l(\mathbf{x}_{l+1}; \theta_l)}{\partial \theta_l} \right)^\top \epsilon_l. \quad (\text{A.10})$$

When g_l takes a particular form we recover a Hebbian-like update rule. Consider a pre-activation

parameterisation of the downward prediction,

$$g_l(\mathbf{x}_{l+1}; \theta_l) = \theta_l \sigma(\mathbf{x}_{l+1}). \quad (\text{A.11})$$

In this case, the parameter update remains local and Hebbian-like:

$$\Delta \theta_{l,ij} \propto \sigma(\mathbf{x}_{l+1})_j \epsilon_{l,i}. \quad (\text{A.12})$$

This can be interpreted as a postsynaptic prediction-error term modulating nonlinear presynaptic activity.

Appendix B. Algorithmic details for tPC-RTRL

All parameter updates in this work follow the forward-update convention of Qi et al. [43]. This method is known to stabilise training in deeper PC networks [43]. For clarity, and compatibility with more general PC works, all definitions are written in the standard free energy form; the forward-update convention does not change the inference procedure and determines only how the parameter derivatives are realised during the M-step. In practice this convention means that during the weight update, rather than evaluating the free energy error terms at the current layer-wise errors, given by the mismatch between the inferred state and its prediction, we evaluate them at the mismatch between the initial feedforward prediction and the inferred state. As an example with the readout error term from the network defined in Figure 2, the standard error term would be

$$\epsilon_y^{(t)} = \mathbf{y}^{(t)} - g(\hat{\mathbf{x}}^{(t)}; \theta_{\text{out}}), \quad (\text{B.1})$$

where $\hat{\mathbf{x}}^{(t)}$ is the inferred latent state. Under the forward-update convention, this becomes

$$\epsilon_y^{(t)} = \mathbf{y}^{(t)} - g(\mathbf{x}^{(t,0)}; \theta_{\text{out}}), \quad (\text{B.2})$$

where $\mathbf{x}^{(t,0)}$ is the value of $\mathbf{x}^{(t)}$ at the start of inference, before any updates to the latent state have been applied. Qi et al. [43] show that this convention keeps the parameter updates anchored to the feedforward trajectory that the model follows during test-time rollout, and therefore improves performance. In practice, the additional cost of evaluating the free energy at the feedforward prediction rather than the inferred state is very small, as it only requires storing the feedforward prediction at the start of inference, and discarding it once calculating the parameter gradients.

Appendix C. RG-LRU implementation details

This appendix further details the Real-Gated Linear Recurrent Unit (RG-LRU) [50] introduced in Section 3.5, this includes the explicit gradients and momentum-accelerated dynamics used during PC inference, the regression-output variant of the per-timestep free energy used on the nanodrone benchmark, and the optional input-projection and learnt state-initialisation heads. Notation follows Section 3.5 throughout where $\mathbf{x}_L^{(t)} \in \mathbb{R}^I$ is the cell input, $\bar{\mathbf{x}}^{(t-1)} \in \mathbb{R}^H$ is the propagated recurrent state, $\mu_x^{(t)}$ is the recurrent prediction, $\mathbf{x}^{(t)}$ is the inferred recurrent latent, and $\mathbf{o}^{(t)}$ is the inferred readout latent.

Input projection. When the input dimensionality I differs from the recurrent state dimensionality H , the input that enters the gated-input pathway of Eq. (24) is projected to $\mathbb{R}^H$:

$$p^{(t)} = W_{\text{in}} \mathbf{x}_L^{(t)} + b_{\text{in}}, \quad W_{\text{in}} \in \mathbb{R}^{H \times I}, \quad b_{\text{in}} \in \mathbb{R}^H, \quad (\text{C.1})$$

and the recurrent update becomes

$$\mu_x^{(t)} = a^{(t)} \odot \bar{\mathbf{x}}^{(t-1)} + \gamma^{(t)} \odot (g_z^{(t)} \odot p^{(t)}). \quad (\text{C.2})$$

The models in Sections 4.2 and 4.3 have $I = H$ and use $W_{\text{in}} = \mathbb{I}$, $b_{\text{in}} = 0$, in which case the projection is no longer a learnable parameter. The nanodrone model (Sections 4.4 and 4.5) uses $I = 4$, $H = 128$ and learns $\{W_{\text{in}}, b_{\text{in}}\}$ jointly with θ_{rec} . In what follows we treat $p^{(t)}$ uniformly, with $p^{(t)} = \mathbf{x}_L^{(t)}$ understood in the un-projected case.

learnt state initialisation. For the nanodrone model (Sections 4.4 and 4.5) the recurrent state at the start of each sequence is produced from the observed body-frame state $s_0 \in \mathbb{R}^9$ by a feedforward head

$$\bar{\mathbf{x}}^{(0)} = \tanh(W_{x_0} s_0 + b_{x_0}), \quad (\text{C.3})$$

with $W_{x_0} \in \mathbb{R}^{H \times 9}$, $b_{x_0} \in \mathbb{R}^H$. The other instances of the RG-LRU model use a fixed zero initial state and do not require this component.

Appendix C.1. PC inference

The full forward pathway at each timestep consists of the recurrent prediction of Eq. (24) followed by a ReLU readout and a linear output head. For the copy task (Section 4.1), WikiText-103 (Section 4.2), and translation (Section 4.3) models the output

head is a softmax classifier with cross-entropy loss, as written in Eq. (25). For the nanodrone model (Section 4.4) the output head is linear and the output term is the squared error

$$\mathcal{F}_{\text{out}}^{(t)} = \frac{1}{2C} \|W_l \mathbf{o}^{(t)} + b_l - \mathbf{y}^{(t)}\|_2^2, \quad (\text{C.4})$$

which replaces the cross-entropy term in Eq. (25). In both cases the recurrent and readout terms are unchanged. The normalising constants $1/H, 1/R, 1/C$ arise from treating each latent as a per-coordinate Gaussian with isotropic precision and absorbing the dimensionality into the coefficient.

Inference is initialised at the feedforward values $\mathbf{x}^{(t)} \leftarrow \mu_x^{(t)}$ and $\mathbf{o}^{(t)} \leftarrow \phi(W_r \mu_x^{(t)} + b_r)$, and minimises $\mathcal{F}^{(t)}$ by a fixed number of momentum-accelerated gradient-descent steps with step size α and momentum coefficient β . Writing the local prediction errors as

$$\epsilon_x^{(t)} = \mathbf{x}^{(t)} - \mu_x^{(t)}, \quad (\text{C.5})$$

$$\epsilon_o^{(t)} = \mathbf{o}^{(t)} - \phi(W_r \mathbf{x}^{(t)} + b_r), \quad (\text{C.6})$$

the gradient of $\mathcal{F}^{(t)}$ with respect to each latent is

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \mathbf{x}^{(t)}} = \frac{1}{H} \epsilon_x^{(t)} - \frac{1}{R} W_r^\top (\epsilon_o^{(t)} \odot m_r^{(t)}), \quad (\text{C.7})$$

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \mathbf{o}^{(t)}} = \frac{1}{R} \epsilon_o^{(t)} + \frac{1}{C} W_l^\top (\hat{\mathbf{y}}^{(t)} - \tilde{\mathbf{y}}^{(t)}), \quad (\text{C.8})$$

where $m_r^{(t)}$ is the ReLU mask from the readout. In the classification case $\hat{\mathbf{y}}^{(t)} = \text{softmax}(W_l \mathbf{o}^{(t)} + b_l)$ and $\tilde{\mathbf{y}}^{(t)}$ is the (optionally label-smoothed) one-hot target; in the regression case $\hat{\mathbf{y}}^{(t)} = W_l \mathbf{o}^{(t)} + b_l$ and $\tilde{\mathbf{y}}^{(t)} = \mathbf{y}^{(t)}$. Each inference step applies

$$v_x \leftarrow \beta v_x + \partial \mathcal{F}^{(t)} / \partial \mathbf{x}^{(t)}, \quad \mathbf{x}^{(t)} \leftarrow \mathbf{x}^{(t)} - \alpha v_x, \quad (\text{C.9})$$

$$v_o \leftarrow \beta v_o + \partial \mathcal{F}^{(t)} / \partial \mathbf{o}^{(t)}, \quad \mathbf{o}^{(t)} \leftarrow \mathbf{o}^{(t)} - \alpha v_o. \quad (\text{C.10})$$

The number of inference steps and values of α, β are task-specific and reported in Appendix G.

Once inference has converged to $(\hat{\mathbf{x}}^{(t)}, \hat{\mathbf{o}}^{(t)})$, the corresponding errors $\hat{\epsilon}_x^{(t)}, \hat{\epsilon}_o^{(t)}$ supply the local signals used to update the readout and output-head parameters via the standard tPC rule $\Delta W = -\eta \partial \mathcal{F}^{(t)} / \partial W$. The recurrent state error $\hat{\epsilon}_x^{(t)}$ is additionally the quantity fed into the RTRL tracker of Appendix C.2 to update the recurrent parameters of the RG-LRU cell.

Appendix C.2. Real-time recurrent learning for the RG-LRU

The recurrent parameters of the RG-LRU cell are $\theta_{\text{rec}} = \{\Lambda, W_a, b_a, W_z, b_z\}$, together with the input-projection parameters $\{W_{\text{in}}, b_{\text{in}}\}$ when the projection is used and the state-initialisation parameters $\{W_{x_0}, b_{x_0}\}$ when a learnt initialisation is used. For each such parameter we maintain an influence tensor $\mathbf{M}_\theta^{(t)} = \partial \bar{\mathbf{x}}^{(t)} / \partial \theta$, updated online after every timestep.

General recursion. Because the cell update of Eq. (24) is element-wise in the hidden dimension, every influence tensor decays multiplicatively by $a^{(t)}$ along its hidden axis. The exact RTRL recursion of Eq. (27), restated here for any $\theta \in \theta_{\text{rec}}$, is

$$\mathbf{M}_\theta^{(t)} = a^{(t)} \odot \mathbf{M}_\theta^{(t-1)} + \frac{\partial \mu_x^{(t)}}{\partial \theta^{(t)}}. \quad (\text{C.11})$$

The element-wise structure of the recurrence means each hidden unit depends only on its own row of W_a, W_z, W_{in} and its own entry of $\Lambda, b_a, b_z, b_{\text{in}}$, so no cross-unit influences need to be stored. The total recurrent storage is therefore $O(H + HI)$ per example, as summarised in Section 5.2.

The immediate term $\partial \mu_x^{(t)} / \partial \theta^{(t)}$ is obtained directly by differentiating Eq. (24) (or Eq. (C.2) when the input projection is enabled); when the projection is omitted, $W_{\text{in}}, b_{\text{in}}$ drop out of θ_{rec} and from the RTRL tracker.

State-initialisation head. For the nanodrone model the initial recurrent state $\bar{\mathbf{x}}^{(0)}$ is produced by Eq. (C.3). Because $\{W_{x_0}, b_{x_0}\}$ enter only at $t = 0$, the influence tensor is initialised once per sequence,

$$\mathbf{M}_{W_{x_0}, ij}^{(0)} = (1 - (\bar{\mathbf{x}}_i^{(0)})^2) s_{0,j}, \quad \mathbf{M}_{b_{x_0}, i}^{(0)} = 1 - (\bar{\mathbf{x}}_i^{(0)})^2, \quad (\text{C.12})$$

and thereafter propagated forward by the multiplicative decay in Eq. (C.11) with no additional immediate term, since these parameters are not reused at $t \geq 1$.

Recurrent parameter gradient. Given the error $\hat{\epsilon}_x^{(t)}$ returned by the inference procedure of Appendix C.1, the tPC-RTRL time-batched recurrent gradient accumulator is

$$\Delta \theta = \frac{1}{H} \sum_{t=1}^T \hat{\epsilon}_x^{(t)} \cdot \mathbf{M}_\theta^{(t)} \quad \text{for each } \theta \in \theta_{\text{rec}}, \quad (\text{C.13})$$

and is applied at the end of the sequence as $\theta \leftarrow \theta + \eta \Delta\theta$, consistent with Algorithm 1. The RTRL recursion, the inference dynamics, and the readout-parameter updates are all applied layer-locally to the RG-LRU cell, and no global BPTT-style reverse traversal through time is required, with spatial error propagation handled by local PC inference steps.

Equivalence with the two-component update. The single-term form above is equivalent to the two-component tPC-RTRL update of Eq. (15) in the predictive-rollout regime. We retain the uncollapsed two-component form in the main text for two reasons. First, it makes the relationship to standard tPC transparent. Standard tPC retains only the immediate term, while tPC-RTRL augments it with the historic term, so separating these contributions cleanly isolates the mechanism responsible for long-range temporal credit assignment. Second, the two-component form is what survives beyond the predictive-rollout regime. Under inferred propagation (Appendix F) the RTRL recursion becomes approximate and the clean collapse into a single term is no longer guaranteed; keeping the two components explicit makes visible where the approximation enters (through the historic term) and which quantity remains exact regardless of regime (the immediate term).

Appendix D. Proof that tPC-RTRL recovers BPTT in the predictive-rollout regime

Millidge et al. [20] showed that, at inference equilibrium and under the fixed-prediction assumption, the parameter updates of a PC network coincide exactly with those of an equivalent network trained by BP. We use this result to show that tPC-RTRL, as specified by the update rule stated in Eq. (15) of the main text, recovers the gradients of a recurrent network trained by BPTT.

We state the result for the single-recurrent-layer setting, which is also the setting in which exact RTRL is most commonly analysed. The extension to multiple recurrent layers is conceptually straightforward, but would require tracking prohibitively large influence matrices. Throughout this proof, we assume the predictive-rollout regime introduced in Section 3, so that the propagated latent state is the recurrent prediction,

$$\bar{\mathbf{x}}^{(t)} = \mu_x^{(t)} = f(\mathbf{x}_L^{(t)}, \bar{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}}), \quad (\text{D.1})$$

and the corresponding RTRL recursion is exact.

Model and objective. Consider a recurrent latent state $\bar{\mathbf{x}}^{(t)}$ together with a depth- L feedforward readout network. Let the readout states be $\mathbf{z}_t^{(1)}, \dots, \mathbf{z}_t^{(L)}$, and define $\mathbf{z}_t^{(0)} \equiv \mathbf{x}^{(t)}$. The output prediction is

$$\bar{\mathbf{y}}^{(t)} := g_L(\mathbf{z}_t^{(L)}; \theta_{\text{out}}^{(L)}), \quad (\text{D.2})$$

and the instantaneous squared-error loss is

$$\ell_t := \frac{1}{2} \|\mathbf{y}^{(t)} - \bar{\mathbf{y}}^{(t)}\|_2^2. \quad (\text{D.3})$$

Under the usual Gaussian assumptions for PC and tPC, the corresponding free energy at timestep t is

$$\begin{aligned} 2\mathcal{F}^{(t)} = & \|\mathbf{y}^{(t)} - g_L(\mathbf{z}_t^{(L)}; \theta_{\text{out}}^{(L)})\|_2^2 \\ & + \sum_{l=0}^{L-1} \|\mathbf{z}_t^{(l+1)} - g_l(\mathbf{z}_t^{(l)}; \theta_{\text{out}}^{(l)})\|_2^2 \\ & + \|\mathbf{x}^{(t)} - f(\mathbf{x}_L^{(t)}, \bar{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}})\|_2^2. \end{aligned} \quad (\text{D.4})$$

We denote the prediction errors by $\epsilon_y^{(t)} = \mathbf{y}^{(t)} - g_L(\mathbf{z}_t^{(L)})$, $\epsilon_l^{(t)} = \mathbf{z}_t^{(l+1)} - g_l(\mathbf{z}_t^{(l)})$ for $l = 0, \dots, L-1$, and $\epsilon_x^{(t)} = \mathbf{x}^{(t)} - \mu_x^{(t)}$.

To recover BP exactly, we adopt the fixed-prediction assumption [20]. That is, the prediction functions and their derivatives are evaluated at the feedforward initialisations and held fixed during inference. We denote these fixed feedforward values by tildes:

$$\tilde{\mathbf{x}}^{(t)} := f(\mathbf{x}_L^{(t)}, \bar{\mathbf{x}}^{(t-1)}; \theta_{\text{rec}}), \quad (\text{D.5})$$

$$\tilde{\mathbf{z}}_t^{(l+1)} := g_l(\tilde{\mathbf{z}}_t^{(l)}; \theta_{\text{out}}^{(l)}), \quad (\text{D.6})$$

$$\tilde{\mathbf{z}}_t^{(0)} := \tilde{\mathbf{x}}^{(t)}. \quad (\text{D.7})$$

In the predictive-rollout regime, the propagated state is exactly this feedforward recurrent prediction, i.e. $\bar{\mathbf{x}}^{(t)} = \tilde{\mathbf{x}}^{(t)}$.

Equilibrium conditions propagate prediction errors through the readout. Inference minimises $\mathcal{F}^{(t)}$ with respect to the latent variables $\mathbf{x}^{(t)}, \mathbf{z}_t^{(1)}, \dots, \mathbf{z}_t^{(L)}$. Setting $\partial\mathcal{F}^{(t)}/\partial\mathbf{z}_t^{(L)} = 0$ gives

$$\epsilon_{L-1}^{(t)} = \epsilon_y^{(t)} \frac{\partial g_L(\tilde{\mathbf{z}}_t^{(L)}; \theta_{\text{out}}^{(L)})}{\partial \tilde{\mathbf{z}}_t^{(L)}}. \quad (\text{D.8})$$

The same argument applied to each intermediate readout state gives the backward-propagating recursion

$$\epsilon_{l-1}^{(t)} = \epsilon_l^{(t)} \frac{\partial g_l(\tilde{\mathbf{z}}_t^{(l)}; \theta_{\text{out}}^{(l)})}{\partial \tilde{\mathbf{z}}_t^{(l)}}, \quad l = 1, \dots, L-1. \quad (\text{D.9})$$

Finally, setting $\partial\mathcal{F}^{(t)}/\partial\mathbf{x}^{(t)} = 0$ (noting $\mathbf{z}_t^{(0)} \equiv \mathbf{x}^{(t)}$) yields

$$\epsilon_x^{(t)} = \epsilon_0^{(t)} \frac{\partial g_0(\tilde{\mathbf{x}}^{(t)}; \theta_{\text{out}}^{(0)})}{\partial \tilde{\mathbf{x}}^{(t)}}. \quad (\text{D.10})$$

Unrolling Eq. (D.9) from $l = L$ down to $l = 0$ and substituting into Eq. (D.10) gives

$$\epsilon_x^{(t)} = \epsilon_y^{(t)} \prod_{l=0}^L \frac{\partial g_l(\tilde{\mathbf{z}}_t^{(l)}; \theta_{\text{out}}^{(l)})}{\partial \tilde{\mathbf{z}}_t^{(l)}} = \epsilon_y^{(t)} \frac{\partial \bar{\mathbf{y}}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t)}}, \quad (\text{D.11})$$

where the last equality is the chain rule through the feedforward readout.

Spatial PC recovers BP at the recurrent state. The instantaneous loss satisfies

$$\frac{\partial \ell_t}{\partial \tilde{\mathbf{x}}^{(t)}} = -(\mathbf{y}^{(t)} - \bar{\mathbf{y}}^{(t)}) \frac{\partial \bar{\mathbf{y}}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t)}} = -\epsilon_y^{(t)} \frac{\partial \bar{\mathbf{y}}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t)}}. \quad (\text{D.12})$$

Combining with Eq. (D.11) gives

$$\epsilon_x^{(t)} = -\frac{\partial \ell_t}{\partial \tilde{\mathbf{x}}^{(t)}}, \quad (\text{D.13})$$

which tells us that the spatial BP error is equal to the PC local error at this inference equilibrium. The same recursive argument also gives the standard spatial-PC result for the readout weights [20],

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{out}}^{(l)}} = \frac{\partial \ell_t}{\partial \theta_{\text{out}}^{(l)}}, \quad l = 0, \dots, L, \quad (\text{D.14})$$

so that PC recovers BP for all parameters in the readout path.

tPC-RTRL update equals the BPTT gradient at each timestep. We now consider the recurrent parameters θ_{rec} . The tPC-RTRL update at time t , as given by Eq. (15) of the main text, is the sum of an immediate and a historic term,

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}} = \underbrace{\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}^{(t)}}}_{\text{immediate}} + \underbrace{\frac{\partial \mathcal{F}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)}}_{\text{historic}}. \quad (\text{D.15})$$

The free energy depends on $\theta_{\text{rec}}^{(t)}$ and on $\tilde{\mathbf{x}}^{(t-1)}$ only through the recurrent prediction $\mu_x^{(t)}$ in the term $\|\mathbf{x}^{(t)} - \mu_x^{(t)}\|^2$. Differentiating gives

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} = -\epsilon_x^{(t)} \frac{\partial \mu_x^{(t)}}{\partial \theta_{\text{rec}}^{(t)}}, \quad (\text{D.16})$$

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}} = -\epsilon_x^{(t)} \frac{\partial \mu_x^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}}. \quad (\text{D.17})$$

In the predictive-rollout regime, $\tilde{\mathbf{x}}^{(t)} = \mu_x^{(t)}$, so the RTRL recursion for the influence matrix is exact:

$$\begin{aligned} \mathbf{M}^{(t)} &= \frac{\partial \tilde{\mathbf{x}}^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} + \frac{\partial \tilde{\mathbf{x}}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)} \\ &= \frac{\partial \mu_x^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} + \frac{\partial \mu_x^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)}. \end{aligned} \quad (\text{D.18})$$

Substituting Eqs. (D.16), (D.17), and (D.18) into Eq. (D.15) gives

$$\begin{aligned} \frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}} &= -\epsilon_x^{(t)} \left[\frac{\partial \mu_x^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} + \frac{\partial \mu_x^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)} \right] \\ &= -\epsilon_x^{(t)} \mathbf{M}^{(t)}. \end{aligned} \quad (\text{D.19})$$

Applying Eq. (D.13) then gives

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}} = \frac{\partial \ell_t}{\partial \tilde{\mathbf{x}}^{(t)}} \mathbf{M}^{(t)}. \quad (\text{D.20})$$

In the predictive-rollout regime, $\tilde{\mathbf{x}}^{(t)} = \tilde{\mathbf{x}}^{(t)}$, and the instantaneous loss ℓ_t depends on θ_{rec} only through the recurrent state at time t . Hence by the chain rule,

$$\frac{\partial \ell_t}{\partial \tilde{\mathbf{x}}^{(t)}} \mathbf{M}^{(t)} = \frac{\partial \ell_t}{\partial \tilde{\mathbf{x}}^{(t)}} \frac{\partial \tilde{\mathbf{x}}^{(t)}}{\partial \theta_{\text{rec}}} = \frac{\partial \ell_t}{\partial \theta_{\text{rec}}}, \quad (\text{D.21})$$

and therefore

$$\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} + \frac{\partial \mathcal{F}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)} = \frac{\partial \ell_t}{\partial \theta_{\text{rec}}}. \quad (\text{D.22})$$

Summing over time. Summing Eq. (D.22) over $t = 1, \dots, T$ gives

$$\begin{aligned} \sum_{t=1}^T \left[\frac{\partial \mathcal{F}^{(t)}}{\partial \theta_{\text{rec}}^{(t)}} + \frac{\partial \mathcal{F}^{(t)}}{\partial \tilde{\mathbf{x}}^{(t-1)}} \mathbf{M}^{(t-1)} \right] &= \sum_{t=1}^T \frac{\partial \ell_t}{\partial \theta_{\text{rec}}} \\ &= \frac{\partial}{\partial \theta_{\text{rec}}} \sum_{t=1}^T \ell_t, \end{aligned} \quad (\text{D.23})$$

which is Eq. (19) of Proposition 1.

Hence, under the fixed-prediction assumption and in the predictive-rollout regime, the two-term tPC-RTRL update supplies the usual spatial backpropagated error signal through the readout network via the equilibrium prediction error $\epsilon_x^{(t)}$, whilst the RTRL recursion composes the immediate and historic terms into the temporal credit-assignment signal through the recurrent latent trajectory. Their sum over time recovers the BPTT gradient.

Appendix E. Cosine similarity between tPC-RTRL and BPTT gradients

The proof in Appendix D establishes that tPC-RTRL coincides with BPTT exactly under the fixed-prediction assumption and converged inference. As discussed in Sections 3.4 and 5.3, the experiments in the main text do not enforce this regime, but instead, for each task, implement iterative free energy minimisation at an operating point selected to maximise the cosine similarity between the tPC-RTRL and BPTT gradients on a single batch of untrained-model data. This appendix reports the cosine similarity between the two gradients over the course of training the tPC-RTRL models on WikiText-103 (Section 4.2), providing an empirical check that the selected operating point continues to produce well-aligned gradients beyond the untrained initialisation at which it was tuned.

At each logged training step, we compute the tPC-RTRL parameter gradient and the BPTT parameter gradient on the same minibatch using the current model parameters, and report the cosine similarity between the two gradient vectors. Both gradients are flattened across all parameters before the cosine is taken. Figure E.8 shows this cosine similarity throughout training. Reported values are means across the 5 training seeds used in Section 4.2. Note that this represents the cosine similarity between the raw gradients, after the Adam scaling step the global cosine similarity pushes even closer to 1.0 as many magnitude discrepancies are absorbed into the per-parameter learning rates.

Appendix F. Training with inferred propagation

In the predictive-rollout regime used throughout the main text, the propagated latent state is the recurrent prediction, $\bar{\mathbf{x}}^{(t)} = \mu_x^{(t)}$, which keeps the RTRL recursion exact and aligns the training and test-time dynamics. An alternative is the inferred-propagation regime, in which the inferred state is propagated forward, $\bar{\mathbf{x}}^{(t)} = \hat{\mathbf{x}}^{(t)}$. This is natural in settings where inference acts as an online state corrector and the aim is to expose the recurrent dynamics to the broader distribution of latent states reachable under inference. However, because the transition from $\bar{\mathbf{x}}^{(t-1)}$ to $\bar{\mathbf{x}}^{(t)}$ is now defined implicitly by iterative inference rather than by a closed-form recurrence, the RTRL recursion is no longer

exact, and the influence matrix must instead be updated via the approximation in Eq. (17).

Appendix F.1. Gradient bias under inferred propagation

The approximation in Eq. (17) replaces the true implicit transition $\hat{\mathbf{x}}^{(t-1)} \rightarrow \hat{\mathbf{x}}^{(t)}$ with derivatives of the recurrent prediction function evaluated at the inferred state. The quality of this approximation depends directly on how far the inferred state deviates from the feedforward prediction. When $\|\hat{\mathbf{x}}^{(t)} - \mu_x^{(t)}\|$ is small, the inferred and predicted transitions are locally similar and the approximation is close to exact. As the deviation grows, the implicit dynamics introduced by iterative inference increasingly diverge from the prediction-function derivatives used by the recursion, and the influence matrix becomes an estimate of the true sensitivity. Consequently, the recurrent parameter gradient computed from the RTRL update appears to acquire a bias that scales with the magnitude of the inference correction.

This dependency is visible empirically in Figure F.9 which sweeps the number of inference iterations on the copy task whilst holding all other hyperparameters fixed¹. Larger numbers of inference iterations produce larger deviations between the inferred state and the feedforward prediction. As this deviation increases, validation loss rises and validation accuracy falls. For very small deviations the inferred- and predictive-rollout regimes are very similar. Beyond a modest deviation threshold, performance degrades, and the approximation-induced bias begins to dominate the learning signal. However, performance still remains higher than that of standard tPC, suggesting that although not perfect, the RTRL term still appears to provide useful temporal-credit information.

Appendix F.2. Inferred propagation on the nanodrone benchmark

Figure F.10 applies the inferred-propagation regime to the nanodrone benchmark of Section 4.4. Under inferred propagation, the training loss drops rapidly and settles well below the predictive-rollout baseline, yet the validation loss remains substantially higher and shows no corresponding decrease

¹These results were obtained by evaluating the RTRL update rule at the inferred value, $\hat{\mathbf{x}}^{(t)}$. Evaluating this update at the predicted state, $\mu_x^{(t)}$, led to very similar results with the same overall trend.

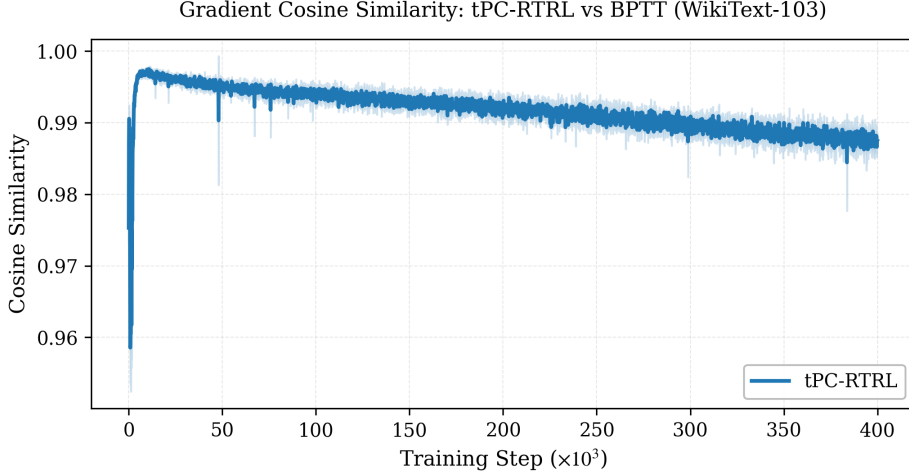


Figure E.8: Cosine similarity between the tPC-RTRL and BPTT parameter gradient vectors on WikiText-103 (Section 4.2), as a function of training step. Both gradients are computed on the same minibatch using the current model parameters. The line shows the mean across 5 seeds; shaded region indicates one standard deviation.

throughout training. Validation is evaluated in pure open-loop rollout without iterative inference, so the validation loss is a direct measure of the autonomous rollout quality of the learnt recurrent dynamics.

We hypothesise that under inferred propagation the latent trajectory followed during training is continually repaired by inference. The recurrent dynamics are therefore never required, during training, to produce accurate predictions from their own autonomous rollouts, and there is correspondingly little optimisation pressure on them to do so. The model is effectively graded on a corrected trajectory that will not be available at test-time, which is related to the issues flagged in Section 3.1. This means that the training loss is optimistic because the model is graded against corrected states, and open-loop rollout performance is poor because the training regime does not incentivise its improvement.

These observations start to help motivate the inference-aware training schedule discussed in Section 5.6. A gradual transition from predictive to inferred propagation during training may allow the model to first establish stable rollout dynamics under the exact RTRL recursion, before being exposed to the richer distribution of latent states reachable under inference. Characterising how tPC-RTRL may perform in this setting is a concrete direction for future work.

Appendix G. Experimental details

Appendix G.1. Learning algorithms

Across all experiments we compare four learning rules that share the same forward model and differ only in how they assign credit to the recurrent parameters.

BPTT. Standard backpropagation-through-time, with gradients propagated through the full recurrent trajectory.

Spatial BP. The recurrent hidden state is detached at every timestep, so only one-step temporal credit assignment is available. Non-recurrent parameters are trained identically to BPTT.

tPC. Standard Temporal Predictive Coding in the predictive-rollout regime. At each timestep the model produces a recurrent prediction $\mu_x^{(t)}$ from the previous propagated state, iterative inference minimises the instantaneous free energy $\mathcal{F}^{(t)}$ to produce the inferred state $\hat{\mathbf{x}}^{(t)}$, and parameters are updated from the corresponding local errors. Recurrent dependencies between timesteps are detached, giving the one-step recurrent update of Section 3.

tPC-RTRL. Identical to tPC in its forward rollout and inference procedure, but the recurrent parameter update is augmented with the RTRL influence-matrix term of Eq. (15), computed exactly via the

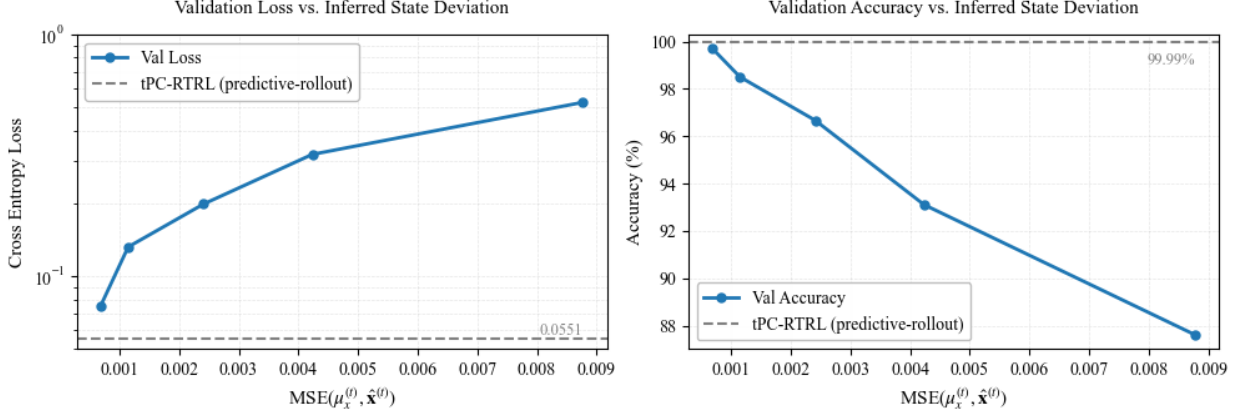


Figure F.9: Effect of inference deviation on copy-task performance under inferred propagation. The number of inference iterations is swept over $\{1, 3, 10, 20, 50\}$ at inference step size 1.0, producing varying mean-squared deviations $\text{MSE}(\hat{x}^{(t)}, \mu_x^{(t)})$ on the horizontal axis. Left: validation cross-entropy loss. Right: validation accuracy. For very small deviations the inferred-propagation regime nearly matches the predictive-rollout baseline, but performance degrades as the deviation grows, consistent with the approximation-quality interpretation of the influence-matrix recursion in Eq. (17).

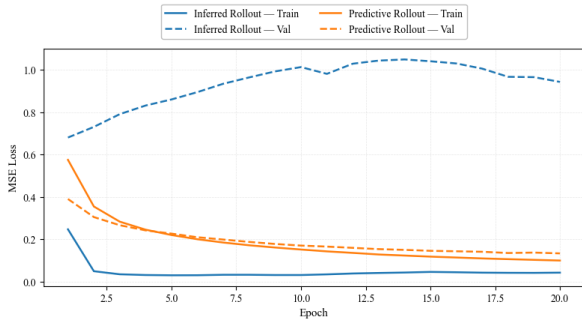


Figure F.10: Training and validation MSE loss on the nanodrone benchmark under inferred propagation (blue) and predictive-rollout (orange). Both models share architecture and optimiser settings; inferred propagation uses 10 iterations with step size 1.0 and momentum 0.9, predictive-rollout uses the setup described in Section 4.4. Inferred propagation achieves substantially lower training loss but markedly higher validation loss. Validation is evaluated in pure open-loop rollout without iterative inference, so the gap reflects a train/test mismatch where inference-based correction during training relieves the recurrent dynamics of the need to produce accurate autonomous rollouts.

recursion in Eq. (16). Non-recurrent parameters are updated as in tPC.

For the two PC-based methods, inference is performed as iterative gradient descent on the free energy with a fixed step size and optional momentum; the number of inference iterations and step size are task-specific and given below. Within each seed all four methods are initialised from the same random parameters, so any difference in performance is attributable only to the learning rule.

Appendix G.2. Copy task

Task. The vocabulary has size 10, with tokens 1–9 sampled uniformly and 0 reserved as padding. Each example consists of a length-30 source sequence and a target shifted by 10 steps, giving a total sequence length of 40. The model is trained with per-timestep cross-entropy to reproduce the delayed sequence, and inputs are represented as one-hot vectors.

Architecture. All four methods share a single-layer RNN with tanh recurrence and a linear readout and hidden size 128.

Training. Models are trained with Adam at learning rate 10^{-3} and batch size 16. One training epoch consists of 16 minibatches, after which the model is evaluated on a fixed validation set of 200 randomly generated samples. The training minibatches for a given seed and epoch are materialised once and

reused across all four methods, as is the validation set. For the PC-based methods we use a single inference iteration with step size 1.0.

Appendix G.3. WikiText-103

Dataset. We use the raw WikiText-103 corpus [28], encoded as UTF-8 bytes over a vocabulary of size 256. Training windows of length 257 are sampled with replacement from the training split; the first 256 bytes form the input and the final 256 bytes the next-byte target. Validation and test windows are constructed deterministically by partitioning the corresponding splits into contiguous non-overlapping windows of the same length.

Architecture. The model consists of a byte embedding of dimension $I = 512$, a single-layer RG-LRU with recurrent state size $H = 512$, a ReLU readout of width $R = 1024$, and a linear output head over the $C = 256$ byte vocabulary. Since $I = H$, the input projection of Appendix C reduces to the identity ($W_{\text{in}} = \mathbb{I}$, $b_{\text{in}} = 0$) and drops out as a learnable parameter. The model has 1,313,536 trainable parameters in total, of which 525,824 are recurrent parameters (the RG-LRU parameters $\theta_{\text{rec}} = \{\Lambda, W_a, b_a, W_z, b_z\}$) requiring temporal credit assignment.

Training. Models are trained for 400,000 steps with batch size 16 using Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$), learning rate 10^{-3} , and gradient-norm clipping at 1.0. A cosine learning-rate schedule with 2000 warmup steps and a minimum ratio of 0.1 is used. For all methods, the embedding layer is loaded from a pretrained checkpoint and kept frozen, isolating the algorithm comparison to the recurrent and readout components. The embedding layer is trained using a single BPTT run from a different seed, the results of this run are not reported in this paper, so do not contribute towards the BPTT results of this task. For the PC-based methods, inference uses 2 iterations with step size 1.0 and momentum 0.9.

Evaluation. Models are evaluated every 5000 steps on validation and test sets. Performance is reported in bits-per-character,

$$\text{BPC} = \frac{\mathcal{L}}{\log 2}, \quad (\text{G.1})$$

where $\mathcal{L}$ is the mean cross-entropy in nats. For each seed the checkpoint with the best validation BPC is used to report final test performance.

Appendix G.4. Translation

Dataset. We use an English–French subset of CCMatrix [29], pre-tokenised and stored as train/validation/test splits of sizes 500,000/50,000/50,000. A BPE tokeniser [52] with vocabulary size 10,000 is trained jointly on the source and target sides of the training split, and the model uses explicit [PAD], [UNK], and [EOS] special tokens. Within each minibatch, source–target pairs are padded to the maximum sequence length in the batch. The data is presented to the model as concatenated source–target sequences, with an [EOS] token marking the boundary between the source prefix and decoder-side target tokens.

Architecture. The architecture mirrors that of the WikiText-103 model at higher width. It consists of a BPE embedding of dimension $I = 1024$, a single-layer RG-LRU with recurrent state size $H = 1024$, a ReLU readout of width $R = 1024$, and a linear output head over the $C = 10,000$ token vocabulary. As in the WikiText-103 model, $I = H$ so the input projection collapses to the identity and drops out of θ_{rec} . The model has 13,399,824 trainable parameters, of which 2,100,224 are recurrent.

Training. Models are trained for 20 epochs with batch size 128 using AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, weight decay 0.01) at learning rate 10^{-3} , a ReduceLROnPlateau schedule, and gradient clipping at norm 2.0. The training objective is token-level cross-entropy with label smoothing 0.05 and padding masked out. For all methods, the embedding layer is loaded from a pretrained checkpoint and kept frozen. The embedding layer is trained using a single BPTT run from a different seed, the results of this run are not reported in this paper, so do not contribute towards the BPTT results of this task. For the PC-based methods, inference uses 2 iterations with step size 1.0 and momentum 0.9.

Scheduled input dropout. To reduce reliance on teacher-forcing, we apply scheduled input dropout. Let the first [EOS] token mark the boundary between source and decoder-side tokens in the concatenated input sequence. From epoch 5 onward, decoder-side input tokens are replaced with [UNK] with a probability that increases linearly over the remainder of training, up to a maximum of 0.3. Source-side tokens are not corrupted.

Evaluation. The final checkpoint is evaluated on the test split. Test BLEU is computed by decoding each target sequence from the source prefix terminated by [EOS], using beam search with beam size 12, length-penalty exponent $\alpha = 1.2$, repetition penalty 1.0, and a maximum decoded length of 128 new tokens. Teacher-forced perplexity is computed with padding masked out.

Appendix G.5. Nanodrone

Dataset and preprocessing. We use the data pipeline released with the Nonlinear System Identification Nanodrone Benchmark [30], which samples synchronised onboard motor commands and motion-capture ground truth at 100 Hz ($\Delta t = 0.01$ s). The training split combines the **random**, **square**, and **chirp** trajectory families (runs 1–3), with run 4 of the same families used for validation. The **melon** trajectory (runs 1–3) is held out entirely as the test set. Trajectories are cut into rolling windows of length $T = 200$ timesteps.

The prediction target is the 9-dimensional body-frame state $s_t = [v_t^b, \phi_t, \omega_t]$ defined in Section 4.4: body-frame linear velocity, orientation as a rotation vector, and body-frame angular rate. Targets are produced by the authors’ preprocessing code as supplied with the benchmark.

Architecture. The model consists of three components. A tanh state-initialisation head maps the 9-dimensional ground-truth drone state to the initial recurrent state $\bar{\mathbf{x}}^{(0)} \in \mathbb{R}^H$ via Eq. (C.3), with $W_{x_0} \in \mathbb{R}^{H \times 9}$ and $b_{x_0} \in \mathbb{R}^H$. The RG-LRU core has recurrent state size $H = 128$ and input dimension $I = 4$, so the learnt input projection $\{W_{\text{in}} \in \mathbb{R}^{H \times I}, b_{\text{in}} \in \mathbb{R}^H\}$ of Appendix C is active and trained jointly with $\theta_{\text{rec}} = \{\Lambda, W_a, b_a, W_z, b_z\}$. The ReLU readout has width $R = 128$ and a linear output head maps to the 9-dim body-frame state. The initialisation head is needed as at the start of the sequence, the drone has some non-zero state that must be mapped into the recurrent state to produce accurate predictions. The overall model has 21,001 trainable parameters.

Training. Models are trained for 50 epochs with batch size 256 using Adam at learning rate 10^{-3} and no weight decay. A **ReduceLROnPlateau** schedule is applied on validation loss with reduction factor 0.5, patience of 10 epochs, and minimum learning rate 10^{-7} . The training objective is the mean-squared

error between the predicted and ground-truth body-frame state, averaged over the batch and over all timesteps in the window. For the PC-based methods, inference uses 3 iterations with step size 1.0 and momentum 0.9, with free energy terms at the recurrent state, the readout projection, and the output all weighted equally. Checkpoints are selected by validation loss.

Evaluation. The selected checkpoint is evaluated by rolling the model out across the full 200-step **melon** windows in open loop, initialising the recurrent state from the ground-truth state at the start of each window. Predicted body-frame velocities are rotated back into the world frame using the predicted rotation vector, and world-frame position is recovered by Euler integration from the ground-truth initial position.

Appendix G.6. Filtering inference iterations

In the nanodrone filtering experiments, test-time correction is performed by running iterative PC inference on the recurrent state whenever an observation is available. Each correction uses 100 inference iterations and an inference learning rate of 1.0. The recurrent prediction at the current timestep is used as the initial latent state and as a prior, while the observed 9-dimensional body-frame state provides the likelihood term. After inference, the corrected latent state is propagated forward, so filtering uses inferred-state propagation at deployment time even though the model was trained in the predictive-rollout regime.